

Contents

1	Introduction	3
2	Bibliographic research	5
2.1	Anatomical basis	5
2.2	Electronics for biopotential measurement	7
2.2.1	Biopotential measurement modes	7
2.2.2	Driven Right Leg	8
2.2.3	Electrode choice	8
2.3	Market research	9
2.4	Noise sources	11
2.4.1	Thermal noise	11
2.4.2	Amplifier and ADC noise	12
2.4.3	Flicker noise	13
2.4.4	ECG noise models	13
2.5	Signal chain design	14
2.5.1	Instrumentation amplifiers	14
2.5.2	Multiplexed ADCs	15
2.5.3	EMI prevention over long wires	15
2.5.4	Common collector voltage buffer	16
2.6	Libio internals	16
3	Analysis, design and implementation	18
3.1	Hardware part	20
3.1.1	Interfacing with the patient	20
3.1.2	ADC	20
3.1.3	Amplification stage	21
3.1.4	Driven Right Leg circuit	24
3.1.5	Microcontroller	25
3.2	Firmware part	26
3.2.1	Serial communication with host	26
3.2.2	Project structure	27
3.2.3	No-OS Drivers	27
3.2.4	libio integration	27
3.2.5	Patching platform drivers for fast UART	28
3.2.6	Continuous and asynchronous data acquisition using IIO triggers . . .	29
3.3	Software part	32
3.3.1	libio	32
3.3.2	Python wrapper class	33
3.3.3	ROS 2 environment	34
3.3.4	ROS 2 implementation	36
4	Results and validation	38

5	Conclusion	42
6	Bibliography	43

1 Introduction

In electrophysiology, two prevalent modes of biopotential measurements are electrocardiography (ECG), which measures cardiac muscle potentials, and electromyography (EMG), which measures skeletal muscle potentials. The most precise measurements are achieved using intracellular electrodes, but are an intrusive method requiring surgical precision and sterile technique. Using surface electrodes is a very common alternative thanks to their non invasive nature, albeit with a loss of spatial specificity, as measurements will contain a superposition of nearby potentials. The field of ECG is by and large applied to identifying and diagnosing cardiac abnormalities in clinical and first responder settings, focusing on disturbances of the cardiac rhythm, artery blood flow, or electrolyte availability for diagnosis of heart conditions. On the other hand, Surface EMG (henceforth sEMG) has broader applications, ranging from the clinical, such as diagnosing nerve and muscle damage, to prosthetics, rehabilitation, biofeedback therapy, ergonomics, sports and movement science [1], and it also opens up the possibility of novel Human-Computer Interaction methods.

Robot Operating System (ROS) is an open-source software framework for robotic applications, providing various mechanisms of interacting with hardware, but most importantly providing a flexible communication system ("the plumbing") that allows for real-time message-passing between networked nodes.[2] In practice, these nodes can be any component of the larger robotic system. This includes a wide range of logical devices, from a discrete piece of hardware to software components. Nodes communicate by sending messages, which can have varying levels of complexity, from primitives to nested aggregated structures, via "topics" which implement publish/subscribe semantics. In our application, a ROS 2 node acts as an abstraction for the hardware, by exposing topics on which it publishes measured samples. Other nodes in the system may subscribe to these topics and use the incoming data for further signal processing.

The Linux Industrial I/O (IIO) subsystem was created in 2008 and mainlined in 2012 (commit 06458e2) to provide a simple and unified way to interact with devices with ADC or DAC capabilities. This category has a broad range including plain ADC/DAC chips, Inertial Measurement Units (Accelerometers, Gyroscopes, Magnetometers), Software Defined Radio, robotic actuators and sensors, and more. Within the Linux kernel, the IIO subsystem fills in a gap between the *input* subsystem which can stream user input data and the *hwmon* subsystem used for monitoring and controlling hardware state such as CPU temperature and fan settings, albeit with low refresh rate. The IIO subsystem provides a modular way to define high bandwidth communication with input/output devices, which are directly connected to the host processor and part of the device tree. This includes devices attached via hardware buses such as SPI, I2C, AXI, etc. Despite the subsystem's name, it is not limited to industrial production applications: IIO usage is ubiquitous in today's linux-based devices for integrating features such as accelerometers, gyroscopes or ambient light sensors in a uniform way, besides other hardware abstraction layers.

Libiio is a software library which provides cross-platform functionality for interacting with both local and remote IIO devices, within the same framework. To accomplish this, it implements the IIO Daemon (IIOD) for which it defines the *IIOD protocol*. The current version of the IIOD protocol (v0.1) is based on a set of human readable commands sent via a single data

stream of alternating client requests and server responses. This allows the same protocol to be used over a variety of transport layers, officially Serial (most relevant for this application), USB and TCP.

Although the electrical design of biopotential measurement devices has been studied for hundreds of years, tracing back to Luigi Galvani’s work in the 18th century, and there exist a number of current-day commercial, academic, and DIY solutions, only a small part aspire to the ideals of Open Source Hardware and/or Open Source Software (ASPEN[3], openBCI[4], BioAmp-EXG-Pill[5], MyoWare EMG[6]), and there exist no systems that integrate with ROS 2 or the Linux IIO subsystem, limiting their flexibility in terms of building software that seamlessly interfaces with them.

We aim to fill this gap and provide an affordable and easily maintainable hardware and software solution for measuring multi-channel biopotential signals and interfacing them with ROS 2 and libiio. Adapting this system to work with other software suites should be made as simple as possible thanks to the versatility of both ROS 2 and libiio, and adapting it to other hardware platforms should also be facilitated by the decoupled nature of the hardware, firmware and software parts. An overview of the information flow, from the biological signals to the software, in figure 1.1.

Further on, we mainly focus our technical effort into designing and implementing the electrical signal chain and the firmware. We implement a lightweight but sufficient software layer which facilitates interaction with the hardware through libiio, a python library, and a ROS 2 publisher node, for other projects to easily build upon.

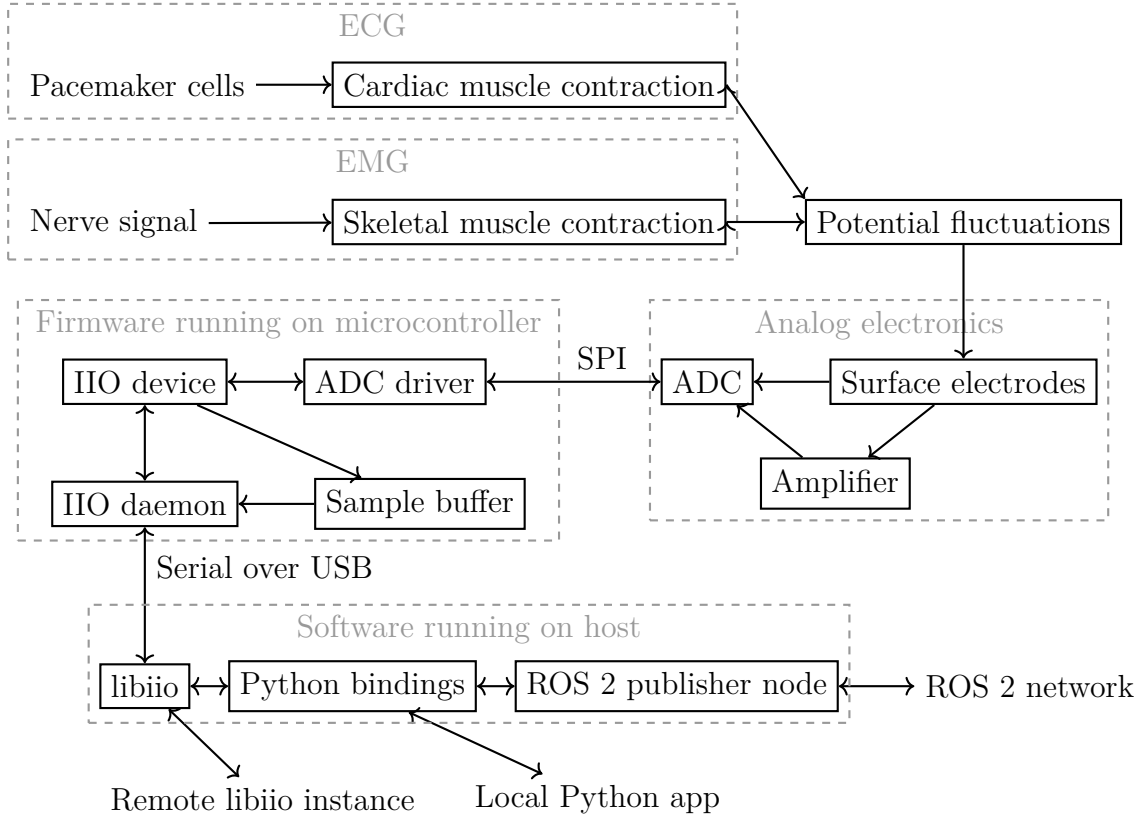


Figure 1.1. Overview of information flow

2 Bibliographic research

2.1 Anatomical basis

As part of normal function, all biological cells have a difference of electrical potential between their interior and their exterior, called the *membrane potential*. In the steady state, the *resting membrane potential* of both skeletal muscle cells and that of cardiomyocytes (cardiac muscle cells) is in the -70 to -90mV range [7], [8].

Skeletal muscles: sEMG

From an electrophysiological standpoint, skeletal muscles are comprised of a number of motor units (MU), each corresponding to an individual axon that innervates a number of muscle fibers. Neural impulses travel from the central nervous system via nerves to a motor unit's innervation zone (IZ), where a number of neuromuscular junction synapses trigger an action potential in the fibers of that motor unit. The electrical activation of the muscle doesn't take place all across the fiber simultaneously, but rather travels from the innervation zone towards the tendons as an impulse.

This starts out as an action potential (AP) at the innervation zone, wherein the span of 4 to 6ms, the membrane potential rises from the resting -70mV to positive (depolarisation) and then back to the resting potential (repolarisation) by biomolecular mechanisms involving the flow of Na^+ , K^+ and Cl^- ions. This localized action potential also triggers action potential in neighbouring parts of the muscle cell, leading to a chain reaction down the muscle fiber with a clear external electrical manifestation: the Motor Unit Action Potential as a traveling impulse of ion flow, which is measurable via the potential fluctuations across tissue.

Muscle fiber contractions follow the "all or nothing" principle, meaning a fiber either contracts fully or not at all. To modulate the force exerted by a muscle, only a part of the available motor units are recruited. Still, there is an independence between the number of motor units, number of fibers, and a muscle's strength, the motor unit count only improving the "resolution" of muscle control. By Henneman's size principle [9], low effort contractions start by recruiting smaller motor units and gradually activate larger motor units as load increases. Besides this spatial control, muscular contraction is also controlled via rate coding, where the same motor unit is reactivated repeatedly, with the activation frequency positively correlating with the overall contraction strength, until fatigue is reached [10].

As a practical result, during low effort motion where only one or a handful of motor units are engaged, the APs are very easily identifiable in the measured waveform. On the other hand, high effort contractions will manifest a superposition of uncorrelated MUAPs which are difficult to separate from just sEMG measurements. (figure 2.1)

Surface measurements have worse spatial resolution and cannot precisely pinpoint the source of the observed electrical activation, but rather "see" a potential halo. In the case of muscle fibers, the surface halo of a travelling action potential is modelled as a directional tripole with a leading positive depolarizing area, a trailing positive repolarizing area, and a strongly negative spike in the middle.[1, Section 3]

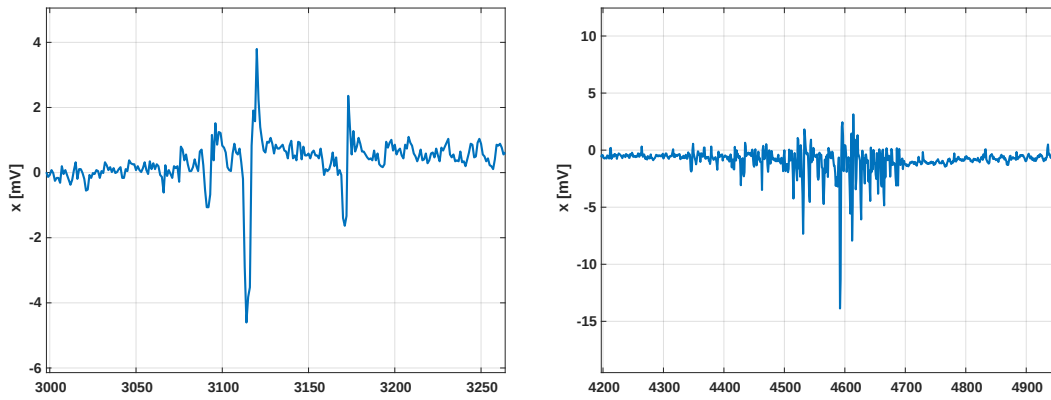


Figure 2.1. Individually identifiable action potentials during a low-effort period (left). Superposition of action potential during a higher-effort movement (right). Electrodes placed 35mm apart on inner forearm.

Cardiac muscles: ECG

While the internal mechanisms of muscle contractions are similar between cardiac and skeletal muscle, their external activation strategies vary almost entirely. Cardiac muscle isn't divided into motor units and as such heartbeat strength cannot be modulated by recruitment, but rather by various chemical and mechanical factors that ultimately increase or decrease the availability of Ca^{2+} ions, such as but not limited to: the Frank-Starling mechanism (which aids in stroke volume regulation), or hormone effects from secretion of adrenaline or norepinephrine.

Contrasted to the usual sEMG measurement, all healthy heartbeats have a clearly identifiable structure consisting of a P wave, the QRS complex, and a T wave. (figure 2.2) Cardiac conditions can be characterized and diagnosed by these features' timing, but their electrical characteristics are very well defined, especially when compared to the noisy-looking MUAP superposition seen during skeletal muscle contraction.

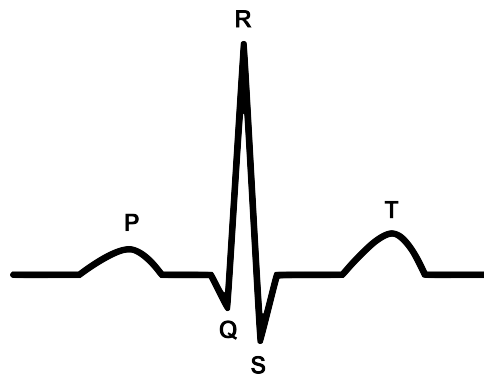


Figure 2.2. Healthy heartbeat electrical signature displaying P wave, QRS complex, T wave. Adapted from Anthony Atkielski under the CC0 / Public Domain license.

Another main difference between sEMG and ECG is electrode placement. While for sEMG we are interested in getting the electrodes as close as possible to the muscle of interest to avoid confusion with other sources of electrical activity, ECG focuses on large scale electric field gradients that appear in the body as a result of a heartbeat. In essence, both the amplitude and "direction" of depolarization and repolarization can be determined by measuring potential differences across a patient's extremities. Localized contractions of cardiac muscle can be identi-

fied by the electric field disturbances they create even tens of centimeters away, bypassing the need to have electrodes as close as possible to the heart. All of this is possible in ECG but not EMG because of the very strong electrical impulses created by the heart, as well as their regular, non-superimposed structure. Various features of the heart's electrical activity can be identified by the directional characteristics of the potential induced in the body, making electrode placement very important for proper ECG.

Over time, conventional electrode layouts for ECG have been developed, aiming to measure well defined features of the heart's function. The standard ECG setup is the 12 lead system based on the following 10 electrodes: RA (Right Arm), LA (Left Arm), RL (Right Leg, usually used as an active electrical reference through "Right Leg Drive"), LL (Left Leg), and the "precordial" V1-V6 electrodes placed in a line arrangement on the chest. From these, 12 electrical potentials (the "leads") are defined and used for ECG diagnosis:[11]

- Lead I, measured as the differential $I = LA - RA$
- Lead II, measured as the differential $II = LL - RA$ (colloquially recognized as "the" heart beat signal)
- Lead III, measured as the differential $III = LL - LA$
- Augmented vector Right, measured as the differential between RA and the short-circuited LA and LL : $aVR = RA - (LA + LL)/2$
- Augmented vector Left, measured as the differential between LA and the short-circuited RA and LL : $aVL = LA - (RA + LL)/2$
- Augmented vector Foot, measured as the differential between LL and the short-circuited RA and LA : $aVF = LL - (RA + LA)/2$
- 6 precordial leads (V1-V6, measured as single-ended signals (referenced to RL , for example, but not necessarily)

The three augmented leads may be computed based on the first three leads, but measurement noise considerations make it more practical to short circuit the averaged leads and thus average their potentials by virtue of the created electrical voltage divider.

2.2 Electronics for biopotential measurement

In essence, biopotential measurements use mechanisms similar to any other electrical time series measurements, but extra design considerations must be taken into account because of the poor electrical characteristics of human bodies (high resistance, parasitic capacitance, easy coupling to nearby interference sources), as well as for patient safety.

2.2.1 Biopotential measurement modes

Two common modes of measurement of biopotentials arise: *unipolar* measurements referred to a common point whose potential isn't affected too much by the observed process, and *bipolar* measurements where consecutive electrodes' potentials are taken into differential amplifiers, as illustrated in figure 2.3. Both are relevant for a biopotential data acquisition system, and as such one of our goals is having a system that can do both.

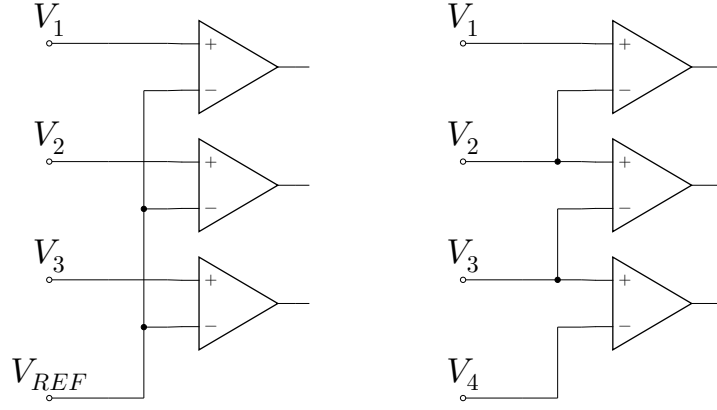


Figure 2.3. Unipolar (left) and bipolar (right) measurements with 4 electrodes

2.2.2 Driven Right Leg

In both measurement modes, having the patient's body properly referred to a known electrical node is very important for the proper functioning of amplification and sensing circuitry. If the patient is "disconnected" from the rest of the circuit, and only interfaced to with high impedance sensing electrodes, stray charges originating from various sources can push the entire patient's body to common-mode potentials above or below the measurable range. Static electricity from clothing can regularly build up hundreds or thousands of volts, which easily exceeds the measurement range of common ADCs or amplifiers. While electrostatic discharge can be accounted for with protection circuitry, such high common-mode voltages make signals unrecoverable. Besides static buildup, a lack of common reference also makes measurements susceptible to picking up mains hum via capacitive coupling between the human body and surrounding mains power lines.

The solution is having a current path from the observed patient to the measurement apparatus' ground, through which charge imbalances can be equalized. This can just be a passive connection to ground, but using active components on this path allows us to modulate the potential difference between the patient and system ground, which opens up the opportunity to both bias all signals to a known voltage, as well as cancelling predictable or low-frequency noise, such as mains hum, by feeding its opposite as a bias (humbucking).

For the practicality of such a biasing circuit, it must connect to the patient in a relatively far away part of the body from what is being measured, so it will minimally interfere with the other electrodes, with as little introduced differential voltages as possible. As per ECG doctrine, the right leg is often used, leading to the name "Driven Right Leg circuit", but this referencing electrode can be connected anywhere depending on the measurement at hand.

2.2.3 Electrode choice

The first step we control in the signal chain is the point of contact with the patient. The electrode type and quality have a great influence on the overall design, performance, and subjective user experience.

In the simplest case, just a metallic electrode connected to the body can sense electrical field changes, forming a polarizable electrode, which cannot offer consistent current flow, but rather

acts like a capacitor, wherein ion concentrations on the skin surface will repel or attract electrons in the electrode, creating small currents.

A much more advantageous setup is one in which the physiological ion currents can be directly converted to electrical current, with an ideal ohmic response. Such an electrode is named a *non-polarizable electrode*. By far the most commonly used electrodes are Ag/AgCl electrodes which fit into this category.[12] These have a silver chloride surface layer, backed by a silver core. Historically, other metals have been used for cost reduction. These are commonly distributed in dry and wet variants. Dry electrodes contact the skin directly, but require an electrolyte solution to be applied for best results. Wet electrodes have the electrolyte solution suspended in an elastic gel that adheres to both the electrode and skin. The electrolyte solution provides free Cl^- ions that facilitate the conversion of electric current to ionic current by ionic decomposition of the AgCl layer or recombining Cl^- ions with Ag^+ ions. Dry electrodes can also somewhat function without any electrolyte solution applied thanks to the dissolved NaCl content of sweat, which also provides free chlorine cations.

Chemically, the AgCl layer partially decomposes into Ag^+ and Cl^- ions when exposed to the aqueous electrolyte solution. When the body potential is higher than the electrode potential, i.e. there's a lack of anions in the body, chlorine cations flow from the electrolyte gel towards the body, further encouraging the decomposition reaction $\text{AgCl} \rightarrow \text{Ag}^+ + \text{Cl}^-$ to replenish them. The positive Ag^+ can precipitate out on the electrode surface, exchanging an electron and thus inducing conventional electrical current "into" the electrode. On the other hand, when the body is more negatively charged than the electrode, Cl^- ions in solution will be in excess and will recombine with Ag^+ ions on the AgCl surface. The new lack of silver anions promotes the ionization of the metallic silver layer, releasing electrons. In both cases, the Ag/AgCl electrode and electrolyte solution act like a mostly resistive interface, with some stray parallel capacitance.

Because of their prevalence, Ag/AgCl electrodes are inexpensive and easily obtainable from general medical suppliers. Additionally, [13] presents 3 methods for fabricating dry Ag/AgCl electrodes, which can be implemented even in small scales and using cheap materials, were these not commercially available.

The state of the art are conductive polymer PEDOT:PSS electrodes. These have great mechanical, electrical, and chemical properties: They adhere and conform to skin, are flexible, self-healing, and have good biocompatibility. They only fall behind the classic Ag/AgCl electrodes in terms of contact impedance, yet are still conductive enough for biopotential readings. They are not commonly available commercially. [14], [15]

2.3 Market research

Biopotential measurements are no new subject and many products already exist both in the academic and commercial fields. All are subject to engineering tradeoffs between parameters such as number of channels, sample rate, precision, noise, electrode types, etc., making each better suited to a certain use case. To the best of our research, there is no universal, modular and extensible solution to the problem of biopotential measurements. Further on, we overview some representative options for EMG / ECG DAQ systems:

OpenBCI

The OpenBCI ("Open Brain-Computer Interface") project started in 2013 and provides open source boards mainly intended for EEG measurements, but which are also applicable for EMG and ECG. These have undergone a handful of revisions in the last decade, and the current generation consists of:

- Cyton – an 8 channel, 24 bit, DAQ based on the ADS1299. Retails for 917 Euro (as of 2024-08).
- Ganglion – a 4 channel, 24 bit, DAQ based on the MCP3912. Retails for 458 Euro (as of 2024-08).
- Daisy – an add-on for the Cython providing 8 extra channels using a second ADS1299. Cython+Daisy package retails for 1835 Euro (as of 2024-08).

While the retail price on the OpenBCI store is steep, design files are provided for all products and they can be ordered and individually built for a fraction of the cost. If built based on the schematics, the bulk of the cost will be the ICs, which add up to around 100 Euro (estimate as of 2024-08).

ASPEN

ASPEN[3] is a project with very similar goals to ours, developed as a PhD thesis at Stanford University, with excellent results, but much more complex than an undergraduate thesis. It is a general-purpose mixed-signal educational system consisting of Arduino shield form-factor boards for filtering, ADC, DAC, DSP, which are then applied to biopotential measurements. The ASPEN paper also contains educational laboratory exercises and didactic materials for teaching mixed-signal design applied to biopotentials.

We aim to design a more minimal system instead of just adapting this, to also comply with the recommendation of using Analog Devices parts.

Quanser QNET Myoelectric board

The Quanser QNET Myoelectric trainer board is a closed-source, one channel add-on board for the National Instruments Elvis II. It provides a fixed amplification stage of powered by a battery to remove power supply noise, and further uses the ELVIS' oscilloscope channel for analog to digital conversion. Its signal chain consists of an overall gain of 300, and first order LPF and HPF limiting the bandwidth to 20 – 500Hz.

Anecdotally, this is the board that sparked the idea for this project, as it is one of the experimental boards given to TUCN students. It depends on the NI Elvis, only has one channel measurements, and is not open source. This makes it a good educational resource, but it falls short on both performance and cost.

BioWare EXG Pills

As one of the cheapest related products, the EXG Pill is an open source instrumentation amplifier board meant to aid in biopotential data acquisition by connecting it to another device with an ADC, typically an Arduino board. It is available for order on the creator's online shop, but being an open hardware project allows anyone to manufacture their own, to specification. It

is not an entire DAQ system, but rather just the amplification stage.

Cometa Pico EMG, Delsys Tringo

As one of the most polished products on the market, the Cometa Pico EMG and Delsys Tringo are two competing multi-channel wireless EMG DAQ systems.

The Cometa Pico EMG supports up to 36 channels. Its sensors have a 2kHz sample rate and an integrated IMU.[16]

The Delsys Tringo has a more flexible system with a range of different sensor types: single bipolar sensors, grid array sensors, snap electrode sensors, which are more intended for high end research. Comparable to the Pico EMG, the Trigno Avanti sensor has a max sample rate of 4370 sps and also have integrated IMUs.[17]

On the other hand, being high-end commercial medical products, both systems cost orders of magnitude more than what we aim for ([18] estimates 15000 Euro for the Cometa and 20000 Euro for the Delsys) and neither abide by the open source philosophy.

MyoWare

Advancer Technologies's line of MyoWare products started out in 2015 with their first generation MyoWare muscle sensor, which similarly to the BioWare EXG Pills described above, only provided the analog signal chain consisting of isolation, an adjustable amplifier and filtering, requiring an external ADC, with the Arduino Uno being recommended for small projects. The current generation has adjustable gain between ? – 200 and a bandpass filter between 20 – 500Hz.[6]

2.4 Noise sources

The superposition of electrical signals measurable by biopotential DAQs contains a complicated mix of noise coming from various physiological and other external factors, besides the signal of interest. Additionally, the measurement circuitry compounds noise at every step. Modeling some of the main noise components of the entire signal chain is essential for its design and proper specification.

Some of the studied noise sources are *intrinsic* to the process and impossible to remove, such as thermal noise and the amplifier and ADC noise profiles. Others are not intrinsic, depend on the setup, and can be compensated for, such as mains hum.

2.4.1 Thermal noise

Even with ideal conductors and measurements, any resistive circuit will be affected by thermal noise, as a consequence of Planck's law of blackbody radiation [19, chapter 3.2]. An ideal resistor R at absolute temperature T will manifest thermal noise V_n over a frequency window Δf having mean squared value $\overline{V_n^2} = 4k_B T R \Delta f$, where $k_B = 1.380649 \cdot 10^{-23} \text{J/K}$ is the Boltzmann constant.

For a rough estimate of the order of magnitude of thermal noise in our biopotential measurements, we study the following hypothetical setup: Assume the electrodes attached to the patient are in thermal equilibrium at a typical skin temperature of $T \approx 308\text{K}$ [20], we are limiting the frequency band to $\Delta f = 500\text{Hz}$ by filtering, and experiencing a typical resistance

$R \approx 3500\Omega$ (electrode + contact + internal resistance) [21]. This is only useful as a rough first order estimation, because the resistance will vary depending on the electrode location, surface preparation, measured individual, electrode condition, etc. The estimated thermal noise is:

$$\overline{V_n^2} = 4k_B T R \Delta f \approx 0.02977 \mu V^2 \quad (2.1)$$

Which can alternatively be interpreted as having an RMS noise of $\sqrt{0.02977 \mu V^2} \approx 172.5 \text{ nV}$ or a noise density of $\sqrt{4k_B T R} = 7.716 \text{ nV}/\sqrt{\text{Hz}}$.

2.4.2 Amplifier and ADC noise

Both ADCs and amplifier circuits have specified noise figures which pose design tradeoffs. In ADCs, the noise is specified as additive noise at the input V_{na} , such that a measurement V_{meas} of a "real" voltage V will be $V_{meas} = V + V_{na}$. Amplifiers are characterized by both input noise V_{ni} created by the input stage, as well as output noise V_{no} created by internal circuitry and the output stage. The output of an amplifier with gain G is $V_{amp} = (V + V_{ni}) \cdot G + V_{no}$. As an easier to use figure, the *Input Referred Noise* is the equivalent disturbance at the input that would create both the effects of input and output noise compounded. $V_n = V_{ni} + V_{no}/G$ such that we can simplify to $V_{amp} = (V + V_n) \cdot G$. An equivalent block diagram is presented in figure 2.4.

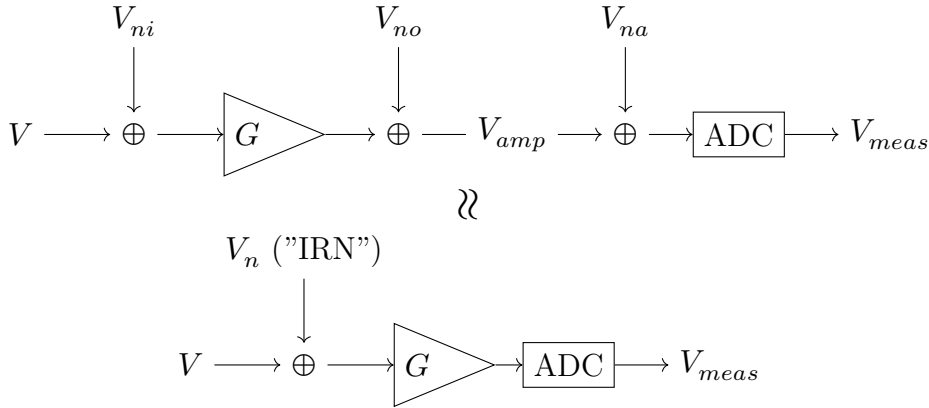


Figure 2.4. Block diagram of amplifier and ADC signal chain, with modelled noise

This raises one of the most important design decisions of the signal chain, namely deciding on the amplification structure before the ADC. Using no amplifier at all is the most economic and also avoids V_{ni} and V_{no} , but in the case of very small input signals, they are left to be dominated by ADC noise V_{na} . Adding an amplifier with unity gain only compounds amplifier noise and is as such detrimental, but further increasing the gain leads to better signal-to-noise ratio (SNR):

$$\text{SNR} = \frac{\text{"signal"}}{\text{"noise"}} = \frac{G \cdot V}{G \cdot V_{ni} + V_{no} + V_{na}} = \frac{V}{V_{ni} + (V_{no} + V_{na})/G} \xrightarrow{G \rightarrow \infty} \frac{V}{V_{ni}} \quad (2.2)$$

Thus, given an amplifier with known V_{ni} we can compute the best-case noise figure assuming "infinite" gain. The rationally convergent nature of the above expression inevitably creates a situation of diminishing returns, wherein increasing the gain will always increase the signal quality, but even relatively small gains achieve practically ideal performance.

The drawback of having very large gain is the limit it incurs on the measurable range of input voltages. Any analog circuit can only handle a limited interval of voltages, which when exceeded leads to unreliable measurements, with the possibility of permanent damage to the equipment. If the ADC is limited to measuring $\pm V_{max}$, the measurable signals are implicitly limited to $\pm V_{max}/G$.

In this topology consisting of a single amplifier stage (with noise V_{ni} , V_{no}), followed by an ADC (with noise V_{na}), a measurement of the "true" voltage V will result in a digitized sample V_{meas} that can be decomposed into:

$$V_{meas} = GV + GV_{ni} + V_{no} + V_{na} \quad (2.3)$$

This relation can be rearranged to obtain an expression for RMS total noise V_n :

$$\overline{V^2} = \frac{\overline{V_{meas}^2}}{G^2} + \underbrace{\left(\overline{V_{ni}^2} + \frac{\overline{V_{no}^2} + \overline{V_{na}^2}}{G^2} \right)}_{\overline{V_n^2}} \quad (2.4)$$

Using this, various combinations of gains, amplifier and ADC characteristics can be compared theoretically, to aid in their selection in the design phase.

2.4.3 Flicker noise

Flicker noise is a low-frequency phenomenon especially prevalent in BJT transistors and devices built using them. It is caused by various material defects which induce fluctuations with power density $P \propto 1/f$. It is also manifested in other active devices (MOSFETs, JFETs) and passive components. It is not a single phenomenon, but rather a mix of various mechanisms, many of which are not fully agreed upon. A practical yet relatively complex solution to flicker noise is modulating the signal of interest to higher frequencies using a chopper and lock-in amplifier. [22]

2.4.4 ECG noise models

Besides the previous intrinsic kinds of noise, further disturbances in ECG measurements can be classified, as per [23], into: low frequency ($< 0.5\text{Hz}$) baseline wander caused by body motion and electrode contact issues, power-line artifacts (50/60Hz and their harmonics) caused by insufficient power supply decoupling when the measurement device is connected to mains power and capacitive or inductive coupling between the patient and surrounding mains lines, and muscle artifacts (EMG signals). Of the three, the first two are relevant and undesirable in any biopotential measurement, while the latter is obviously the signal of interest in EMG applications.

From a signal processing standpoint, a simple solution for low frequency wander and power line artifacts is using a DC blocking comb filter, but these can also be addressed in the hardware design, especially when it comes to power supply decoupling.

2.5 Signal chain design

With today's available technology, signals in the millivolt range are relatively easy to measure even at quite high rates, if strongly driven. Intrusive methods of biopotential acquisition such as needle electrodes plugged directly into muscle fibers can work with no amplification stage, and just a direct connection to the ADC board, while still having good signal quality. On the other hand, when measuring with surface electrodes, both the electrode and biological tissue become a very high impedance element in the signal chain, which means that even relatively high biopotentials will have a hard time overcoming parasitic effects and signal chain noise before reaching the ADC.

2.5.1 Instrumentation amplifiers

Instrumentation amplifiers are a class of differential amplifier which boast high open-loop gain, low noise, and high input impedance. The typical textbook realization is with 3 operational amplifiers connected like in figure 2.5, which allow the gain to be adjusted by choosing the value of resistor R_G . Their high input impedance means that very little current is needed to affect the inputs, helping with the issue of high electrode and tissue impedance. Their high power supply rejection ratio (PSRR) and common mode rejection ratio (CMRR) make them even more suitable for differential electrode applications, requiring very little extra circuitry to eliminate power line noise coming both from the electronics themselves, as well as coupled to the patient.

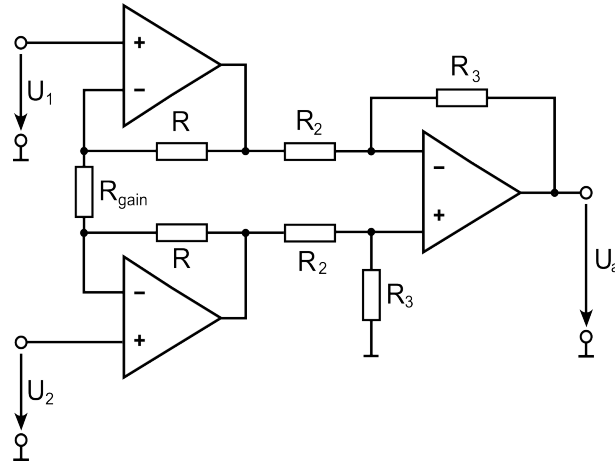


Figure 2.5. Typical implementation of an instrumentation amplifier. Adapted from wdw under the CC BY 2.5 license.

In this setup, the amplified output is described by equation 2.5. R_{gain} is adjustable and practically used to set the gain of the amplifier. The other resistances are not usually specified exactly, but rather just by ratio.

$$U_a = (U_1 - U_2) \cdot \underbrace{\left[\frac{R_3}{R_2} \cdot \left(1 + \frac{2R}{R_{gain}} \right) \right]}_G \quad (2.5)$$

2.5.2 Multiplexed ADCs

Analog to Digital Converters come in a great variety of classes and specifications, depending on their intended use. The class most suitable for our use-case are multiplexed ADCs, which have a single internal measurement element, and multiplex one of many inputs to it to effectively do multiple channel time series measurements. The main characteristics used for selection are: the number of channels, output data rate (ODR), sample FIFO (or lack thereof). For good signal-chain design, we must also carefully consider internal clock performance and independence from the serial clock, digital filtering capabilities, channel multiplexing dead time and other noise characteristics. We explain each of these further.

The number of channels and **ODR** are intuitively relevant to the performance of the system, directly impacting the characteristics of the resulting time series.

Some multiplexed ADCs have internal **sample FIFOs** which can accumulate measured samples and which allow the host to read multiple queued up measurements instead of having to poll each sample as it becomes available. While it does not have an effect on signal quality, having an internal FIFO relaxes firmware timing constraints.

Many multiplexed ADCs contain **integrated digital filters**, often not bypassable. These help with signal quality and are typically used for rejecting unwanted frequency bands, especially powerline noise. While they increase single-channel performance, they often introduce **multiplexing dead time**, wherein a filter must process a number of "useless" samples to enter its steady state regime after a channel switch. This dead time can greatly reduce overall data rate to even orders of magnitude less than the maximum data rate when iterating through all channels.

Sample acquisition timing is essential and has great impact on the resulting signal. Clock jitter results in phase noise due to aperture uncertainty, and it directly limits SNR according to equation 2.6[24], where f represents signal frequency and t_a represents RMS clock jitter in seconds-RMS.

$$\text{SNR} < -20 \log(2\pi f t_a) \quad (2.6)$$

ADCs which depend on the serial clock for their internal logic (such as the AD5592, AD5593) will automatically suffer from clock jitter issues and we must disqualify them and add a strict requirement for independence between the communication clock and internal ADC logic clock.

Finally, various internal phenomena introduce noise dependent on the measurement configuration, with a typical increase of noise with increasing measurement rate. Datasheet-specified noise figures should be consulted for the chosen parts.

2.5.3 EMI prevention over long wires

When dealing with long wires and weak signals, electromagnetic interference (EMI) is a significant source of noise which is produced by virtually all electronic devices around. To combat this, common techniques are using twisted pairs and wire shielding. For simplicity, we will try to shorten the wires carrying weak signals as much as possible and only use strongly driven and high amplitude signals over long wires. The result is that in bipolar measurements, we

place the differential amplifier as close as possible to the electrodes, and then don't care about signal degradation between the amplifier and ADC thanks to the strongly driven nature of that signal.

2.5.4 Common collector voltage buffer

Later used in both the preamplifier and Driven Right Leg circuits, the common collector amplifier (illustrated in figure 2.6) is a minimal and cheap circuit for providing a low impedance, high drive strength "mirror" of an incoming signal, which may have too high impedance to be used by itself.

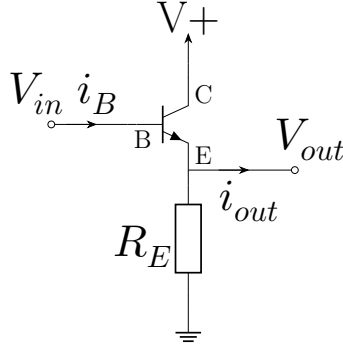


Figure 2.6. Common-collector voltage buffer

Assuming the input voltage puts the transistor in the linear operating range, the output voltage will lie around:

$$V_{out} = V_{in} - V_{BE} \approx V_{in} - 0.7V \quad (2.7)$$

The emitter resistor R_E provides negative feedback, modulating the emitter current i_E so the circuit tends towards the equilibrium point given by:

$$R_E \cdot (i_E - i_{out}) = V_{out} = V_{in} - V_{BE} \quad (2.8)$$

The transistor's amplification allows the output port to source up to $i_{out} \leq \beta \cdot i_B$, and the circuit can sink up to $i_{out} \geq -V_{out}/R_E$. This boundary condition is given by the case in which the transistor is at the limit between off and conducting.

2.6 Libiio internals

In essence, libiio[25] facilitates communication with IIO-like systems from software. A system containing multiple IIO devices is the *iio context*. All iio contexts provide the same functionality and structure, but they may vary in the communication model, abstracted through their *backend*. The main backends are: `local` for "true" Linux IIO devices defined through the device tree, `serial` for libiio devices that communicate via serial port, `usb` for wired devices with a libiio-specific USB stack, `ip` for networked communication with a libiio daemon. Contexts contain a list of *devices*, each of which containing a list of *channels*. Both contexts, devices and channels can have a number of *attributes* for configuration, which are string key - string value

pairs. Channels are meant for sending or receiving sampled data, whose storage and transfer is done via *buffers*. Each channel buffer can specify its direction (input/output/both) and data format (signedness, bits per sample, bit layout).

The two main interactions with a libiiio device are reading/writing attributes and reading/writing sample buffers. Attributes are interacted with in simple request/response transactions. Sample data transfers are more involved, requiring allocation of buffers, and exchanging their contents in chunks.

All libiiio communication, regardless of underlying transport layer, is done via the IIOD protocol, which is a human readable command-reply protocol. It defines 14 commands: HELP, EXIT, PRINT, VERSION, TIMEOUT, OPEN, CLOSE, READ, WRITE, READBUF, WRITEBUF, GETTRIG, SETTRIG, SET, of which the following are of especial importance to our device:

- OPEN <device> <samples_count> <mask> - starts acquisition on the specified device, using samples_count as the buffer size, and mask as a bitmap of which of the available channels to use
- CLOSE <device> - stops acquisition on the specified device
- READBUF <device> <bytes_count> - reads buffered data from a device doing acquisition
- READ/WRITE - read and write device/channel attributes

Libiiio daemons are software which allows communicating with a libiiio context between otherwise incompatible backends. This project uses two libiiio daemons: the first one resides in firmware and handles communication between a "native" device and a libiiio serial transport, and the second resides in software and further exposes the serial transport to the network, for easier low-privilege use and remote access of the device.

3 Analysis, design and implementation

We set some objectives for the biopotential data acquisition system and categorize them into MUSTs, which are absolutely required for success, SHOULDs, which can go to future work if unchecked, and vaguer non-checkable guiding principles:

The project MUST:

- Consist of **Open Hardware and Software** under a permissive license, making all design files available for free and allowing anyone to use, modify, and improve them.
- Allow for **at least 8 measurement channels**
- Measure with **at least 600Hz sample rate** to unambiguously capture EMG components up to 300Hz
- Have **high CMRR, PSRR**, to the point where powerline noise is either not present or can be easily filtered out digitally
- Have sufficiently **high SNR**: $> 40\text{dB}$ for EMG, $> 20\text{dB}$ for ECG
- Provide software support for interfacing with **ROS 2 and libiio**

The project SHOULD:

- Allow for **16 measurement channels**
- Allow per-channel **configurability of differential / single-ended measurements**
- Provide active **Right Leg Drive**
- Measure with **at least 1000Hz sample rate**, ideally a power of two (1024, 2048) for "round" FFT bins of 1, 0.5Hz
- Measure with **1uV precision**

Lastly, some guiding principles are cost reduction, utilisation as an educational tool, modularity (decoupled hardware - firmware - software), and use of Analog Devices parts, to be later usable as a product demo.

We split the project into 3 main domains: mixed-signal, embedded firmware, and software.

The **mixed-signal domain** consists of the electrodes, wires, amplification stages, and ADC. It is the most sensitive part of the signal chain, requiring care in its design and construction to bring as little noise as possible into the measured signals.

The **embedded firmware domain** represents the code running on a microcontroller that communicates with both the host computer and the ADC. It will be implemented as a serial libiio device, allowing it to be configured from the host computer, for it to later record ADC data, buffer it, and send it in chunks. It is less sensitive but must still respect a number of timing and data integrity guarantees for proper function.

The **software domain** will be a minimal layer on top of libiio to provide easy access to the device via Python 3 bindings and a ROS 2 node. These two are chosen so that other software may be easily interfaced with this project.

The overall assembled setup consisting of a MAX78000FTHR microcontroller, EVAL-AD4114-SDZ ADC assembly, Right Leg Drive and Amplifier boards, is presented in figure 3.1.

Lastly, a minor convenience was designing and 3D printing a mounting bracket for the components, helping with immobilizing them and preventing accidental short circuits from loose wires.

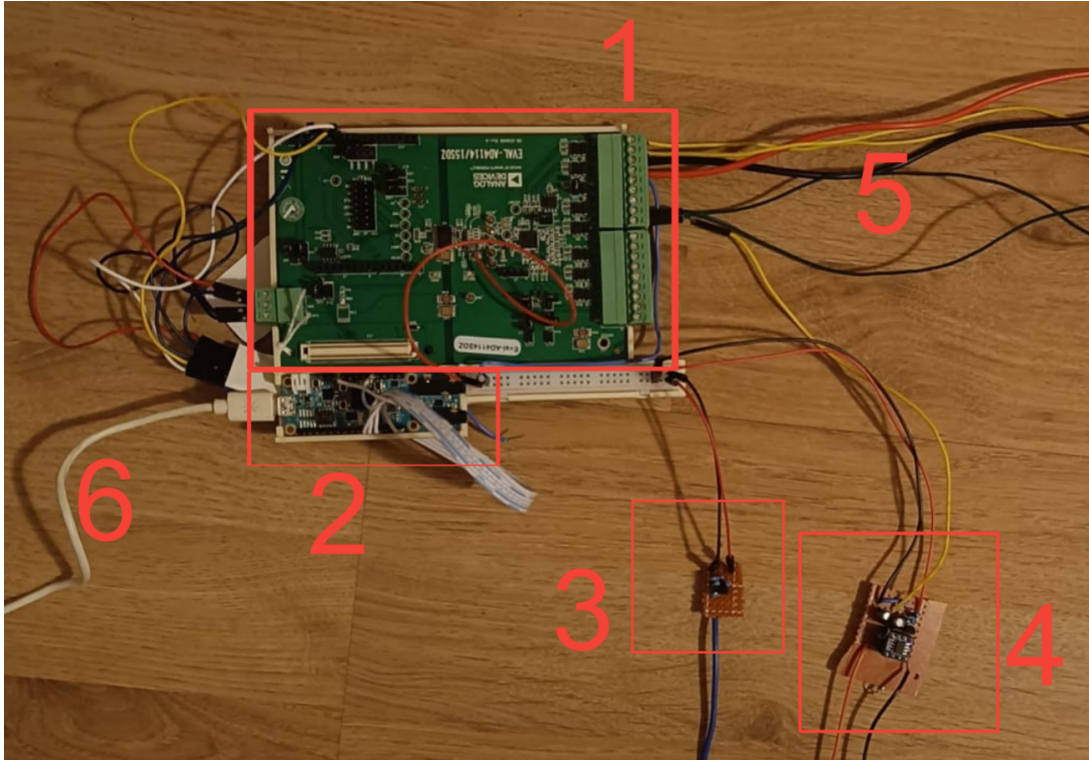


Figure 3.1. Assembled project setup with highlighted components: (1) EVAL-AD4114-SDZ, (2) MAX78000, (3) Right Leg Drive board, (4) AD8226 instrumentation amplifier board, (5) electrode wires, (6) USB to host computer

3.1 Hardware part

3.1.1 Interfacing with the patient

Taking from the research from the previous electrode type research, we do not consider PEDOT:PSS electrodes to be viable for this project due to them not being commercially available in standard and mass produced forms, being mostly a custom product experimented with in today's laboratories, even half a century after their discovery in the 1980s.

We settle for snap-on adhesive Ag/AgCl gel electrodes thanks to being cheap, readily available, standardized, and they can be connected to with inexpensive alligator clips, besides special (but more expensive) snap clips. The downsides are their comparatively larger contact area (cca 200mm² vs [1]'s 2mm electrode diameters), and are lacking in subjective user experience, sometimes being very painful to detach.

3.1.2 ADC

We initially chose the AD7124-8[26] thanks to its very diverse feature set, including: programmable gain amplifiers (PGAs) which had the potential to remove the need for the external amplification stage, high precision, high number of channels, and high sample rate. Unfortunately, it is not a single cycle ADC. This means that although it has very good throughput on a single channel, when switching between two channels there is a dead time for the integrated filter to settle, which effectively reduces the per-channel sample rate to less than 100Hz when sampling all 16, which is virtually useless for ECG ($f_{\max} = 150\text{Hz}$) and EMG ($f_{\max} = 500\text{Hz}$).

Our next choice was the AD4114[27]. It doesn't include PGAs, but otherwise has very good specifications and has a much higher sample rate and a single cycle conversion mode. Switching ADCs was facilitated by the modular nature of the project.

Moreso, to avoid the need for PCB design, we use the EVAL-AD4114SDZ evaluation board, which already implements isolation between the digital and analog sides, overvoltage and ESD protection, an external voltage reference, and various other features.

The AD717X family of ADCs, which the AD4114 is part of (in spite of the numbering scheme), all expose a similar interface to the application developer. A number of channels can be enabled, which are then measured in a loop, in ascending order of their indices, by use of an internal sequencer. Each of the 16 channels can select one of 34 inputs: 16 unipolar measurements ($V_{in,k} - V_{incom}$), 16 bipolar measurements ($V_{in,2k} - V_{in,2k+1}$ and $V_{in,2k+1} - V_{in,2k}$), an internal temperature sensor or the voltage reference itself. If an input is selected by multiple channels, it is read multiple times in the sequence. To parametrize a channel's conversion, one of 8 "setups" can be assigned. Setups describe whether a measurement is unipolar or bipolar, the used reference, buffering and filtering settings. Each setup has a corresponding digital filter with an internal state. If multiple channels refer to the same setup, they will also be alternately using the same digital filter, requiring some extra logic for avoiding latent (i.e. "hidden") samples from passing from one channel's to another's readouts. Conveniently, the SING_CYC flag puts the ADC in "single cycle mode", which always flushes out the digital filter when switching channels, at the expense of worse filter performance, data rates, and greater noise.

The AD4114 lacks an internal sample FIFO, meaning that data from a conversion must be read

Table 3.1. Selected measurement configurations and their specifications

N	ODR (Hz)	Filter	Per-channel rate (Hz)	Noise ($\mu\text{V RMS}$)
1	1007	sinc5+sinc1	1007	20
2	2597	sinc5+sinc1	1298.5	38
3	3115	sinc5+sinc1	1038.33	41
4	4444	sinc5+sinc1	1111	56
5	5181.65	sinc5+sinc1	1036.33	64
6	6211	sinc5+sinc1	1035.17	74
7 – 10	10309	sinc3	$10309/N > 1000$	1068
11 – 16	10309	sinc3	$10309/N < 1000$	1068

before the next one or else it is lost. While the digital interface allows for an interrupt signal on the $\text{DOUT}/\overline{\text{RDY}}$ line to notify the host of new data, it is incompatible with most microcontrollers' hardware SPI implementations and it is much more suitable for an FPGA host. Therefore we can and should resort to polling the status register to check for new samples, which are announced through the RDY bit. Polling should be done at least as fast as the conversion rate. Experimentally, we deduced a 5MHz limit for SPI communication, in spite of the theoretical 20MHz limit of the AD4114, likely due to parasitic effects on uneven length jumper wires for high speed SPI.

One of the most important limitations in determining which parameters to run the ADC in is its noise, which varies with the sample rate. While high sample rates are very desirable given we have practically limitless digital signal processing capacity, they also degrade the signal quality. This is especially relevant when sampling multiple channels, where the Output Data Rate (ODR) is divided between them. A single channel measurement with $\geq 500\text{Hz}$ bandwidth can be done with the 1007Hz setting for an effective resolution of 17.3 bits, but having 6 channels forces the use of the 6211Hz setting with 15.4 bits of resolution, and more channels than that start limiting the bandwidth to less than 500Hz.

For this purpose, we selected 7 configurations depending on the number of required channels so as to maintain at least 1000Hz per-channel sample rate (table 3.1). Measurements of up to 6 channels (single-ended or differential alike) can use the better performing sinc5+sinc1 filter. Measurements of 7 – 10 channels cannot achieve the desired sample rate without switching to the faster yet more noisy sinc3 filter. Measurements of 11 – 16 channels cannot achieve the desired sample rate in any configuration using the AD4114, with the max per-channel sample rate degrading to $10309/16 = 644.3125\text{Hz}$ for the maximum of 16 channels.

3.1.3 Amplification stage

The chosen AD4114 ADC has $N = 24$ bits of resolution with one LSB equating to the following in the bipolar measurement mode.

$$\text{LSB} = V_{REF} / (2^{N-1} \cdot 0.1) \quad (3.1)$$

Using the internal 2.5V reference, $\text{LSB} \approx 2.98\mu\text{V}$. The noise level varies depending on the

sample rate. At the single-channel $f_s = 1007\text{Hz}$, 6 channel $f_s = 6211\text{Hz}$, and all channel $f_s = 10309\text{Hz}$, the ADC noise is specified to be around $\overline{V_{na,1007}^2} = 20\mu\text{V}$, $\overline{V_{na,6211}^2} = 74\mu\text{V}$ and $\overline{V_{na,10309}^2} = 1068\mu\text{V}$ RMS respectively. Without an amplification stage, this would result in the following SNRs relative to a 1mV signal typical of ECG/EMG:

$$\begin{aligned} \text{SNR}_{(\text{dB})} &= 10 \log_{10} \left(\frac{(1\text{mV})^2}{\overline{V_{na}^2}} \right) \\ \text{SNR}_{1007} &\approx 34\text{dB} \\ \text{SNR}_{6211} &\approx 23\text{dB} \\ \text{SNR}_{10309} &\approx -0.6\text{dB} \end{aligned} \tag{3.2}$$

In the case of 7+ channels, a 1mV signal slightly dips under the noise floor, becoming unmeasurable. Thus, a preamplification stage is required for enabling the use of all channels.

We choose the AD8226 [28] instrumentation amplifier as the central component of the preamplification stage. It is an inexpensive, readily available instrumentation amplifier with more than sufficient CMRR ($> 80\text{dB}$). It can be configured for a gain of $G = 1$ to 1000 using a resistor $R_G = 49.4\text{k}\Omega / (G - 1)$. Its RMS voltage noise for our desired bandwidth of $\Delta f = 500\text{Hz}$ is:

$$\begin{aligned} \sqrt{\overline{V_{ni}^2}} &= \sqrt{500\text{Hz} \cdot (22\text{nV}/\sqrt{\text{Hz}})^2} = 0.492\mu\text{V} \\ \sqrt{\overline{V_{no}^2}} &= \sqrt{500\text{Hz} \cdot (120\text{nV}/\sqrt{\text{Hz}})^2} = 2.683\mu\text{V} \end{aligned} \tag{3.3}$$

With the preamp stage added before the ADC, the total noise of a sample and the theoretical SNR are:

$$\begin{aligned} \text{SNR}_{(\text{dB})} &= 10 \log_{10} \left(\frac{(1\text{mV})^2}{\overline{V_n^2}} \right) \\ (2.4) \Rightarrow \text{SNR}_{(\text{dB})} &= 10 \log_{10} \left(\frac{(1\text{mV})^2 \cdot G^2}{\overline{V_{ni}^2} \cdot G^2 + \overline{V_{no}^2} + \overline{V_{na}^2}} \right) \end{aligned} \tag{3.4}$$

As described in section 2.4.2, increasing the gain will increase SNR, but with diminishing returns, and it will limit the available measurement range. For the AD8226, we compute a handful of gain scenarios (table 3.2) and choose $G \approx 20$ as a fair middleground between good SNR and good input range, allowing for some fluctuations without saturating the sensors.

We use the part's gain formula to compute a suitable gain resistor (equation 3.5). The closest E6-series value is $R_G = 2200\Omega$ which would give a gain of $G \approx 22.45 \approx +27\text{dB}$. This increases the resolution from approximately $3\mu\text{V}/\text{LSB}$ to $3/22.45 \approx 0.13\mu\text{V}/\text{LSB}$

$$R_G = \frac{49.4\text{k}\Omega}{G - 1} = \frac{49.4\text{k}\Omega}{20 - 1} \approx 2600\Omega \tag{3.5}$$

Table 3.2. Full scale, SNR for various gains

Gain	Approx. range	SNR
1	5V	34dB
2	2.5V	40dB
5	1V	48dB
10	500mV	54dB
20	250mV	59dB
50	100mV	64dB
100	50mV	65dB
1000	5mV	66dB
∞	0	66.16dB

Using an active element (the instrumentation amplifier) also gives us the opportunity to do some hardware filtering. To combat high frequency noise, we add a first order RC low-pass filter with a cutoff of $f_c \approx 500\text{Hz}$. Our implementation uses $R = 330\Omega$ and $C = 1\mu\text{F}$ for a theoretical 3dB cutoff frequency of:

$$f_c = \frac{1}{2\pi \cdot 330\Omega \cdot 1\mu\text{F}} = 482.28\text{Hz}$$

To encode the bipolar differential measurement as a unipolar signal, we bias the amplifier's output voltage using the AD8226's V_{REF} input. To generate the bias voltage, we use a $10\text{k}\Omega - 10\text{k}\Omega$ voltage divider from 5V to ground, resulting in 2.5V. As specified in the datasheet, this pin must be driven with less than 2Ω impedance, so this wouldn't suffice. We use a common-collector voltage buffer to amplify the divider's drive strength, biasing the output to approximately $V_{REF} \approx 5\text{V}/2 - 0.7\text{V} = 1.8\text{V}$ after the PN junction voltage drop. To help with transient power supply fluctuations, the instrumentation amplifier's positive supply as well as V_{REF} are decoupled with $0.1\mu\text{F}$ capacitors. The second should also reduce BJT flicker noise coming from the voltage buffer.

The overall circuit diagram and a picture of the built amplifier board are presented in figure 3.2. Experimental validation of the preamplifier board is covered later in the paper.

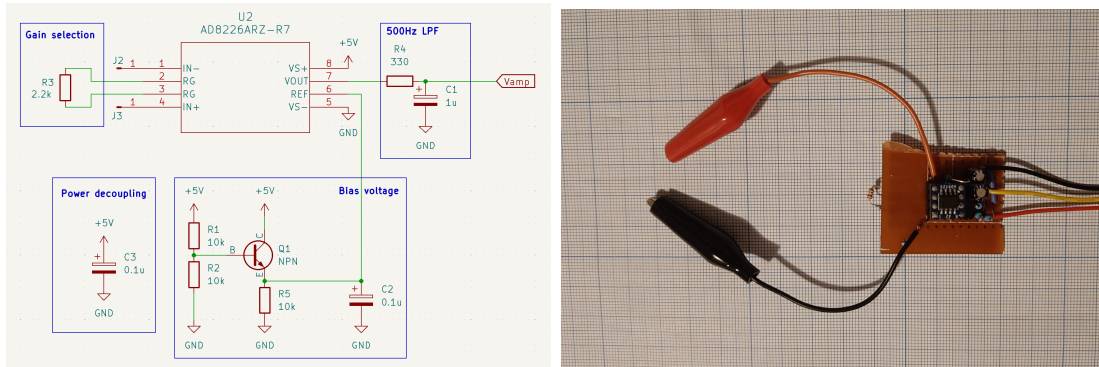


Figure 3.2. Circuit diagram and photo of built amplifier board

3.1.4 Driven Right Leg circuit

Because the ADC and measured patient can be considered to be practically galvanically isolated (very high impedance between them), it is important to have a well defined potential difference between them, or else measurement circuitry may saturate. In the case of the AD8226 based preamplifier board, the common-mode voltage of its inputs must be in the 0.7 to 3V range for its output to have the full rail-to-rail range. The AD4114 is more forgiving, with an input range of $\pm 20V$, but it still requires a common reference.

To satisfy these constraints, we build another buffered voltage divider circuit, with a $10k\Omega - 20k\Omega$ divider producing approximately $5V \cdot (20/30) = 3.33V$ and a common collector buffer which drops the voltage down to $3.33V - 0.7V \approx 2.63V$ but which is strongly driven. A $1k\Omega$ emitter resistor means this circuit can sink up to $2.63mA$, which is more than enough for an active Driven Right Leg circuit, but low enough not to create a risk of electrocution.

The bias voltage of $2.63V$ above analog ground is applied to the patient and brings common mode voltages all across the body in an acceptable range for both the AD8226 preamplifier board and the AD4114 ADC.

The active nature of the DRL circuit means it actively cancels out coupled powerline noise and baseline wander, as illustrated in figure 3.3. This is opposed to passive referencing to ground, which in our experiments mitigated powerline noise but not baseline wander.

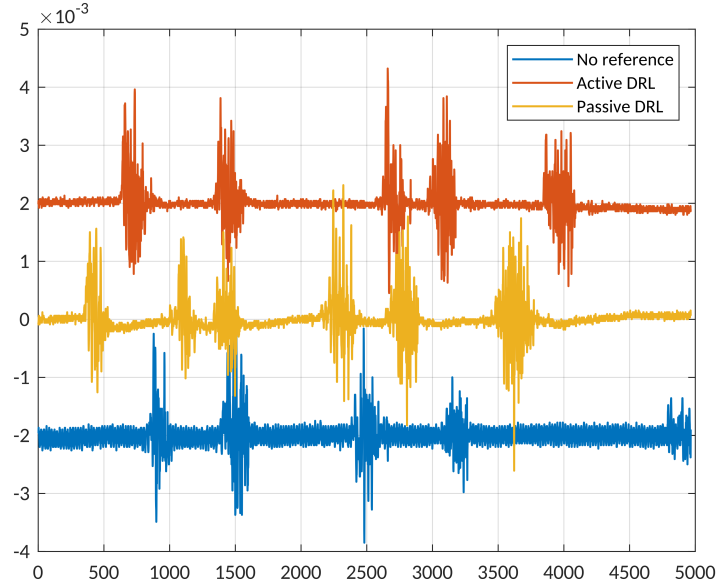


Figure 3.3. Measured differential EMG with no reference (blue), passive referencing to ground (yellow), active DRL (red). DC offset is artificially added for illustration

The circuit schematic for the Driven Right Leg circuit and a photo of its implementation are shown in figure 3.4.

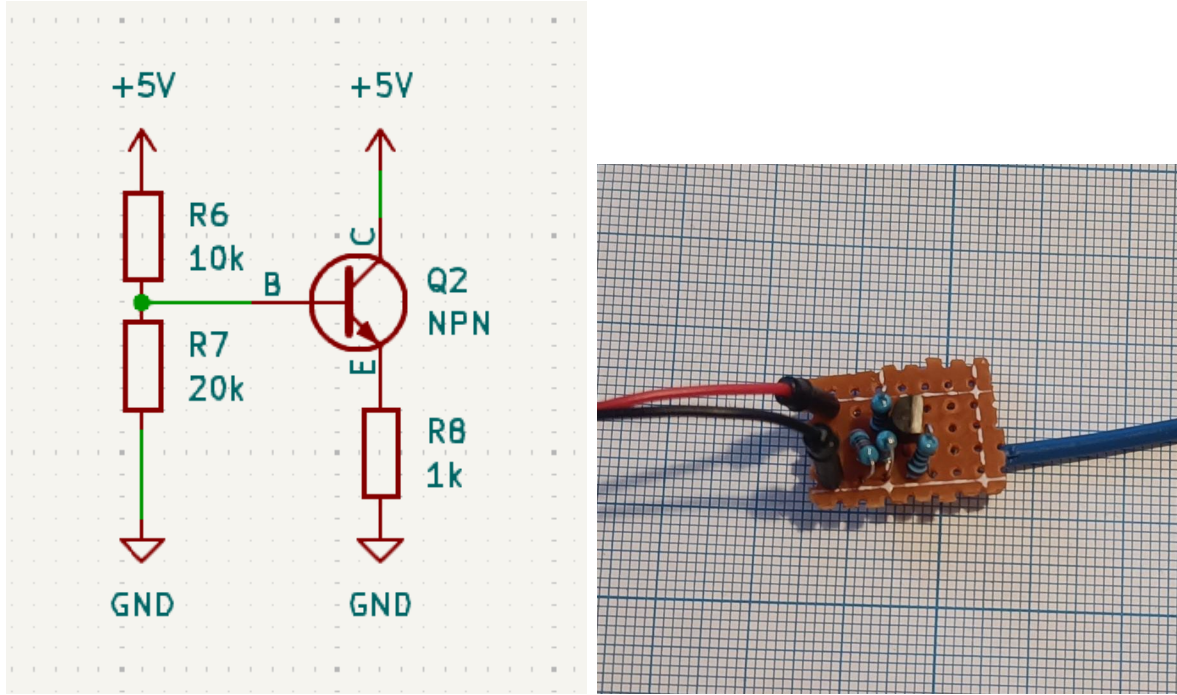


Figure 3.4. Schematic of DRL circuit (left) and built DRL circuit (right)

3.1.5 Microcontroller

For microcontroller selection, the main performance considerations are on the one hand the ability to communicate via SPI fast enough to comply with the desired sample rate, and on the other hand for it to have a fast enough clock rate and memory for the IIO Daemon which will comprise most of the final application. For further work, networking capabilities would be a nice-to-have to be able to run a standalone ROS 2 node. A plethora of well known boards match these criteria, among which the very popular Espressif ESP8266 and ESP32, or the Raspberry Pi RP2040 and RP2350 would be sufficient and good alternative choices thanks to their excellent community support. The popular, yet more and more obsolete ATMEGA328p of Arduino fame isn't fast enough for both recording and further transferring all 16 channels, especially not through unmodified IIO Daemon / ros-micro implementations.

We chose the Analog Devices **MAX78000FTHR microcontroller platform** thanks to its powerful specs which would also allow for some degree of digital signal processing on the board itself. We will use the MAX78000's dedicated SPI bus which is exposed on the feather board. On board, it also has a MAX32625 acting like a USB interface, which also exposes DAPLINK functionality, allowing for easy firmware uploads (by just copying files onto a virtual mounted filesystem) and debugging (via CMSIS-DAP by using the VS Code environment with the Cortex Debug extension).

3.2 Firmware part

For the firmware, we will be structuring our code around No-OS[29], an open-source firmware framework developed by Analog Devices that defines a common API for accessing common peripherals on embedded systems. While its scope somewhat overlaps with a typical Hardware Abstraction Layer (HAL) within an Operating System, no-os focuses on providing minimal abstractions to run directly on bare metal, as the name suggests. Additionally, it provides abstractions for the platform architecture, build systems and custom vendor SDKs, such that code can be very easily ported from one platform to another.

The benefits of using such a versatile framework were constantly apparent during this project's development, which required us to change the microcontroller platform, peripherals, as well as communication protocols during prototyping. While each change required small rewrites of the codebase, the uniform structure of no-os meant we didn't have to rewrite thousands of lines of code after each hardware change.

3.2.1 Serial communication with host

In order to send data to the attached computer or carrier board, the de facto solution is using UART-over-USB, which exposes an USB interface to the attached host and UART to the microcontroller. Most commonly, microcontrollers have dedicated circuitry for UART transmit and receive, which allows the rest of the program to run concurrently to serial communication. This is implemented using two FIFO buffers, one for data pending to be sent from the microcontroller (TX FIFO) and one for received data ready to be processed by the program (RX FIFO).

De facto, most microcontroller development boards, including the MAX78000FTHR, and the more common ATMEGA368P "Arduino-like" boards provide UART-USB translation via a dedicated IC, abstracting away the complexities of the USB communication stack.

UART modules have varying degrees of configurability of the baud rate, flow control, and framing. For most practical uses, though, including this project, one may just specify the baud rate, and default to 8N1 (8 data bits, no parity bit, 1 start bit, 1 stop bit) framing and no flow control. Typical baud rates include 9600 baud, a legacy of 20th century POTS modems (introduced by the V.32 ITU-T standard) and 115200 baud, common both in last-century modem systems, but also recently popularized by its default use in the Arduino development environment.

To achieve the required bandwidth for this project, a suitable baud rate must be chosen. Given the output data rate ODR, number of bytes per channel sample d , and 8N1 framing (giving 10 baud/byte), the baud rate f_{baud} must be:

$$f_{baud} \geq \text{ODR} \cdot d \cdot 10 = 1024 \cdot 10309 \cdot 4 \cdot 10 = 412360 \text{ baud} \quad (3.6)$$

Protocol overhead means we must choose a baud rate safely above it. Choosing a baud rate larger than this minimum will also allow for less timing concerns, especially when it comes to not overflowing the receive and transmit buffers.

Baud rate selection is not universal across devices and the firmware for each hardware platform must choose a rate that is both compatible with the hardware and that satisfies relation 3.6. In

the case of the MAX78000FTHR, a rate of exactly 491520 baud can be achieved by using the IBRO (Internal Baud Rate Oscillator) of 7.3728MHz with a clock divider of 15, but we will use a rounder 500000 baud for convenience.

3.2.2 Project structure

No-OS recommends the following project structure, which we follow:

```
project
├── src
│   ├── common: declarations common to all platforms
│   │   ├── common_data.h
│   │   └── common_data.c
│   ├── platform: platform-specific declarations
│   │   ├── maxim
│   │   │   ├── parameters.c
│   │   │   ├── parameters.h
│   │   │   └── platform_src.mk: brings in platform-specific sources
│   │   └── platform_includes.h
│   ├── iio_ad4114_exg.c
│   ├── iio_ad4114_exg.h
│   └── main.c
├── Makefile: refers to no-OS makefiles
└── src.mk: list of source files
```

The separation of common and platform specific parameters is relevant especially when it comes to initialization of the UART, SPI, IRQ, timer subsystems, where there are usual parameters common to any platform, such as baud rates, SPI mode (CPHA, CPOL), timer frequency, etc, but also platform drivers may specialize some, such as choosing a specific IRQ controller and priorities, SPI drive strength, UART flow control, etc.

3.2.3 No-OS Drivers

Fortunately, the no-OS ecosystem already provides drivers for both the MAX78000 platform as well as the AD4114 as part of the AD717X family of ADCs[30]. This means that we do not have to implement the SPI digital communication protocol ourselves, as the most common operations: initialization, selecting channels and inputs, assigning measurement setups, reading data, are already implemented.

3.2.4 libiio integration

It is quite common for no-OS projects to provide IIO functionality, and for this reason no-OS has a dedicated libiio protocol implementation. This provides a convenient way of describing libiio contexts, with customizable callbacks for all read/write operations with both actual sample data and attributes. Additionally, it implements an internal event loop which abstracts away asynchronous communication with both the host computer and the connected peripherals, which is a major concern in a system where concurrent real-time communication takes place.

The first step in defining the structure of the IIO context is, as it solidifies a mapping between the device-specific functionality on the hardware side and the application-specific functionality on the software side. There is a spectrum of choice between exposing attributes for very low-level details (such as individual flags in registers), which offloads most application logic

to the software side, and exposing more high-level attributes which are translated into register values by the firmware. Because we are not defining a general-purpose ADC device, we will restrict ourselves to only one configurable attribute per channel, namely the desired input to measure, opaquely choosing the best suited sample rates, filter settings, and setup assignment based on table 3.1. Read-only attributes will expose the computed sampling frequency for the device, as well as calibration values (scale and offset) and a list of available input pairs for each channel.

De facto, attributes with values selectable among a set (such as the channel input pair selection) are named by appending `_available` to the name of the attribute they refer to. In our case, channels have a read/write attribute input for querying or setting the selected input pair, as well as the read-only `input_available` which contains a space-separated string of all input pair names.

For lower-level functionality and debugging, an IIO debug interface is also specified, which allows for direct manipulation of the ADC's registers. This shouldn't be normally used, but allows for total control of the device in case the user needs it.

Besides setting up the IIO device, we must also initialize an "IIO app", which is the daemon itself, which may communicate with multiple devices. The IIO app is initialized through the `iio_app_init_param` struct passed to the `iio_app_init` function, which contains: a list of all devices, UART parameters, a list of hardware triggers.

We must also allocate a buffer for samples to be stored in between them being submitted by the ADC and them being read by the host computer. In our implementation, this is a static fixed-size buffer in the global namespace, with room for 1024 samples on 16 channels, for around one second buffer time:

```
char _iio_ad4114_samples_buf[16 * 4 * 1024];
```

3.2.5 Patching platform drivers for fast UART

One encountered issue specific to this setup (no-OS IIO, Maxim microcontroller, using MSDK) was that the internal implementation of asynchronous UART receive operations was very inefficient. It is based on a high amount of indirection, wherein multiple logical layers implement the ability to reconfigure the handling of incoming data at runtime. Both the underlying Maxim SDK (msdk), the no-OS MAX78000 platform driver, and the no-OS IIO implementation added their own software-defined interrupt handling logic based on an excess of virtual function pointer tables, dynamic memory allocation, and unnecessary control structures. All of these slowed down UART communication enough for the firmware not to be able to satisfy the high throughput we specified.

The main symptom of these inefficiencies is that an "UART byte receive" interrupt would take so long to execute that the next byte would arrive before it is done. This would result in the NVIC (Nested Vector Interrupt Controller) launching a nested instance of the same interrupt handler within itself, breaking application logic.

Prior to our modification, the interrupt handler for receiving a byte from UART was a long and convoluted chain with multiple indirect calls:

UART0_IRQHandler → _uart_common_handler → MXC_UART_AsyncHandler → MXC_UART_RevA_TransactionAsync, queueing a no-OS callback to uart_rx_callback → max_uart_read_nonblocking which indirectly calls max_uart_callback which stores the incoming byte to a buffer. Needless to say, this is an *extreme* amount of overhead for what is essentially passing an 8 bit value from the UART receive register to the no-OS UART read buffer.

```
// Initial IRQ handler
void UART0_IRQHandler()
{
    _uart_common_handler(MXC_UART0);
}
```

We propose a lightweight modification which bypasses this entire mechanism and replaces all of it with just a very quick 8 bit store to a circular buffer. In case error bits are set in the UART registers, we fall back to the original implementation, which handles edge cases. For storing queued bytes, we use a lf256fifo, which is an efficient 256 byte circular buffer implementation provided by no-OS:

```
// Modified IRQ handler
void UART0_IRQHandler()
{
    // Short-circuit complex logic
    bool fallback = true;
    if(g_uart0_rx_fifo &&
        (MXC_UART0->int_fl & MXC_F_UART_INT_FL_RX_THD))
    {
        // Got one byte. Clear interrupt:
        MXC_UART0->int_fl |= MXC_F_UART_INT_FL_RX_THD;
        // Store byte in fifo
        lf256fifo_write(g_uart0_rx_fifo, MXC_UART0->fifo);
        fallback = false;
    }

    // fall back to slow handler if there was an unexpected
    state:
    if(fallback)
        _uart_common_handler(MXC_UART0);
}
```

3.2.6 Continuous and asynchronous data acquisition using IIO triggers

By default, the expected behaviour of IIO devices is to start acquisition on selected channels when requested via an OPEN command, and stop and disable the channels once the requested number of samples is reached. In low sample rate scenarios, the host may request the next acquisition window very soon after the current one finishes, and assuming the time delay between windows can be minimized with respect to the overall window size, this could prevent missing any samples and only introduce a small amount of phase drift. Additionally, there

are high-speed applications in which continuity between acquisition windows is not strictly required. On the other hand, with our roughly 10kHz sample rate (cca. 100 μ s/sample) and requirement to not miss any samples, this application has very little room for error in timing. As per [31], modern real-time linux setups have scheduler latencies in the tens and hundreds of microseconds, which realistically leaves no time to actually receive the current window's buffer and send a new acquisition request without missing any samples, even if the acquisition channels were left enabled.

With this general issue in mind, IIO defines triggers as software or hardware events that initiate data acquisition without requiring regular polling from the host. Data captured as a result of triggers is stored in circular buffers and provided with minimal latency whenever the host polls next.

With the AD4114 in continuous conversion mode, two trigger sources are available: using the DOUT/ $\overline{\text{RDY}}$ signal to get an interrupt when a new sample is ready, or using a timer to poll the AD4114 at least as fast as the conversion rate. While the first option is more elegant and wouldn't do any unnecessary polling, it requires nonstandard operation of the SPI protocol by keeping CS active and monitoring the MISO (DOUT/ $\overline{\text{RDY}}$) line. This isn't generally possible, unless we implement the SPI protocol in software, or have access to more configurable IO options, such as on FPGA platforms, or using the Programmable Input and Output subsystem of RP2040 MCUs ([32]). The second option of setting up a timer interrupt which polls the AD4114 status register to check if a new conversion result is available and easy to implement.

To compute a suitable interval for the polling frequency f_p it is trivial that $f_p \geq \text{ODR} = 10309\text{Hz}$. Additionally, we cannot poll faster than it takes to do the polling itself, which is an SPI transfer of 2 bytes (1 request byte, 1 response byte with the status register), amounting to $T = 2 \cdot 10/f_{\text{SPI}} = 4\mu\text{s}$ and limiting $f_p \leq 250\text{kHz}$. There is no use in polling orders of magnitude faster than samples become available, so we settle for polling at around $1.5\times$ the sampling ratio, with $f_p = 16000\text{Hz}$. In the highest data rate scenario, we will have a sample available every $16000/10309 \approx 1.55$ polls on average. In the lowest data rate scenario, this number rises to $16000/1007 \approx 15.89$ polls per sample.

The regularly timed polling is done by using an IIO hardware trigger based on a timer IRQ. When any channels are active, the timer IRQ is enabled by the underlying IIO implementation and its handler can poll, get data, and store it in a buffer to be sent to the host later. Because of the stateless nature of these interrupt requests, we must add extra logic to remember how full each of the channels' buffers are. For this, at the beginning of the acquisition, each channel gets assigned an offset into the main buffer, such that enabled buffers' data will not overlap. The list of enabled buffers is passed as a bitmask in mask.

```
iio_dev->last_enabled_channel = -1;
int k = 0;
for(int i = 0; i < 16; i++)
{
    if(mask & NO_OS_BIT(i)) { // channel i is enabled
    {
        ad717x_assign_setup(dev, i, k%8);
```

```
        iio_dev->channel_offset[i] = k++;  
        iio_dev->last_enabled_channel = i;  
    }  
}
```

The `last_enabled_channel` value is useful in determining when the channel sequencer rolls over from the last channel to the first. Besides the time required for communication, storing the samples should also be done as quickly as possible to not take up microcontroller execution time. In our implementation, this is done by just a single 32 bit store in a static sample buffer. When successfully polling and reading a new sample, its value is stored in the sample buffer in the correct position based on the previously computed channel offset, and a check of whether the sample buffer is full and ready to submit (i.e. the last sequenced channel has just been read) is done:

```
// part of the poll timer interrupt:  
status = data & 0xff;  
channel = status & 0xf;  
data = data >> 8; // remove status least significant byte  
  
iio_dev->sample_buf[iio_dev->channel_offset[channel]] = data;  
  
// Got a full sample of all channels, push it to the buffer  
if(channel == iio_dev->last_enabled_channel)  
{  
    iio_buffer_push_scan(dev_data->buffer, iio_dev->sample_buf)  
    ;  
}
```

3.3 Software part

3.3.1 libiio

With a working serial IIO device developed in the previous section, we can connect to it with any libiio compliant application without requiring further development.

For instance, `iio_info` provided by the `libiio-tools` package can display the structure of described IIO context, with the context-device-channel hierarchy, their specific attributes, and channel specifications:

```
$ iio_info -u serial:/dev/ttyACM0,500000
iio_info version: 0.25 (git tag:v0.25)
[... snip ...]
IIO context has 2 devices:
  iio:device0: ad4114-exg (buffer capable)
    16 channels found:
      voltage0: ch0 (input, index: 0, format: le:u24/32>>0)
        5 channel-specific attributes found:
          attr 0: input value: VINO-VINCOM
          attr 1: input_available value: [... snip ...]
          attr 2: raw value: 9246832
          attr 3: scale value: 0.0029802322387
          attr 4: offset value: -8388608
      voltage1: ch1 (input, index: 1, format: le:u24/32>>0)
        [...snip...]
      voltage15: ch1 (input, index: 1, format: le:u24/32>>0)
        [...snip...]
```

For most of the hardware and firmware development, we used `iio-oscilloscope` to interact with the device. It allows us to inspect and possibly modify attributes, and perform time-series measurements (figure 3.5).

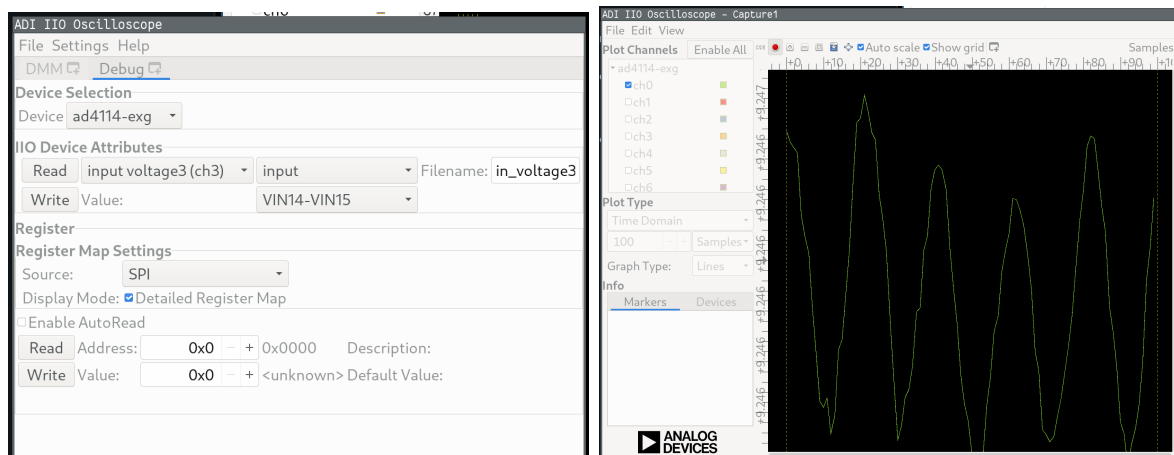


Figure 3.5. Using `iio-oscilloscope` to inspect and change attributes (left), and plot measured samples (right)

3.3.2 Python wrapper class

For programatic interaction with the device, we create a Python 3 package defining a device class that exposes functionality for connecting to the device, configuring its channels, starting acquisition, receiving samples, and stopping acquisition.

We extend the `pyadi-iio` library, which is itself a wrapper around the `pylibiio` bindings for `libiio`. While the bindings themselves would suffice, they expose low level `libiio` details which are unnecessary for this project. An example snippet of how this package can be used is presented:

```
from ad4114exg_iio import ad4114exg_iio

# Connect to libIIO device based on URI
uri = 'serial:/dev/ttyACM0,500000'
dev = ad4114exg_iio(uri)

# Start acquisition of the following inputs
dev.start(['VIN13-VINCOM', 'VIN2-VIN3', 'VIN3-VIN2'])

# Query sample rate, based on the number of inputs
fs = dev.sample_rate
print(fs)

# Receive 100 windows of measured data and print
for i in range(100):
    window = dev.rx()
    print(window)
```

Within `pyadi-iio`, common ADC and DAC functionality such as channel selection, buffer creation, buffer transfers, are clustered into the `tx`, `rx`, and `tx_rx` classes, which most devices inherit from. Additionally, connecting to a `libiio` context is done via the `context_manager` class. The most relevant library functions we use are:

- `_set_iio_attr` and `_get_iio_attr` for reading and writing device and channel attributes, such as the selected inputs
- `rx_buffer_size` for controlling the size of received buffers. Larger buffer sizes result in rarer and bulkier transfers (within the limits of the firmware buffer), and smaller buffer sizes result in having transfers happen more often with smaller payloads. We set a size of 16 samples / transfer for an estimated $1000\text{Hz}/16 \approx 60\text{Hz}$ buffer transfer rate.
- `rx_enabled_channels` selects enabled channels and starts data acquisition upon being written to

Taking from other ADC implementations, we define a device class `ad4114exg_iio` inheriting from `rx` and `context_manager`. The class constructor is largely boilerplate for the proper initialization of the `rx` superclass. Further, we define the channel structure using a nested `_channel` class describing the available attributes:

```
class _channel(attribute):
    def __init__(self, ctrl, channel_name):
        self.name = channel_name
        self._ctrl = ctrl

    @property
    def raw(self):
        return self._get_iio_attr(self.name, "raw", False)

    @property
    def scale(self):
        return float(self._get_iio_attr(self.name, "scale",
False))

    @property
    def offset(self):
        return float(self._get_iio_attr(self.name, "offset",
False))
```

The most important part of the wrapper class is the channel configuration and acquisition start function. It takes an array of strings as a parameter, representing one of the 34 available channel inputs (e.g. "VIN3-VINCOM"). For a provided array of N input strings, the start function assigns them in order to channels $0 \dots N - 1$ and enables these first N channels. The buffer size is set to 16 samples/channel/buffer, equating to roughly $1000\text{Hz}/16 = 62.5$ transferred buffers per second, with variation depending on the exact data rate imposed by the number of channels.

```
def start(self, inputs):
    for i, inp in enumerate(inputs):
        self._set_iio_attr(f"voltage{i}", "input", False, inp)

    nchan = len(inputs)
    self.rx_buffer_size = 16
    self.rx_enabled_channels = list(range(nchan))
```

Overall, given the amount of logic implemented "behind the scenes" by pyadi-iio, creating this functional wrapper required minimal coding, with more than half of the file being boilerplate.

3.3.3 ROS 2 environment

The ROS 2 framework is notorious for difficulties in managing systems based on linux distributions that are not officially supported. When working with ROS 2, it is most practical to use a virtual machine or container to isolate the ROS 2 environment from the host linux distribution, as package version mismatches make it very difficult to run otherwise.

For this project, we prepare a docker environment so that this software may be runnable on any

linux distribution. Starting from the OSRF ROS 2 iron image, we further install libiio, plotjuggler, and the pyadi-iio python package. The following dockerfile is minimal but sufficient:

```
FROM osrf/ros:iron-desktop-full

RUN apt-get update && \
    apt-get install -y \
        libiio-dev libiio-utils \
        ros-iron-plotjuggler-ros \
        python3-pip
RUN python3 -m pip install pyadi-iio

WORKDIR /ros2_ws
```

We build a docker image and instantiate it with an interactive terminal (-it), load the ros environment, and then run any necessary ros2 commands:

```
$ cd thesis/software
$ docker build . -t ros2iio
$ docker run -it -v ./ros2_ws -v /tmp/.X11-unix:/tmp/.X11-unix ros2iio
# source /opt/ros/iron/setup.bash
# ros2 pkg list
action_msgs
action_tutorials_cpp
action_tutorials_interfaces
[...snip...]
```

In the previous commands, we mounted the project's software workspace (-v ./ros2_ws) and the /tmp/.X11-unix file to enable X11 forwarding for ROS 2 GUI apps such as plotjuggler.

Within the ROS 2 workspace, we create 3 packages:

- ad4114exg_interfaces defining application-specific messages
- ad4114exg_publisher containing the publisher node
- ad4114exg_iio providing the previously implemented Python bindings

The first package uses the ament-cmake build system, to allow for message definitions, while the other two use ament-python:

```
# ros2 pkg create --build-type ament_cmake ad4114exg_interfaces
# ros2 pkg create --build-type ament_python ad4114exg_publisher
# ros2 pkg create --build-type ament_python ad4114exg_iio
```

3.3.4 ROS 2 implementation

The chosen data format for biopotential measurements is a timestamped float32, defined as the message `ExgSample.msg`:

```
std_msgs/Header header
float32 value
```

While the header field adds a small size overhead due to the unused frame id, it helps with consistency and interoperability with other ROS 2 projects, it being the de facto standard to timestamp messages. For instance, `plotjuggler` assumes timestamped data to have a `header.stamp` field.

Communication with the serial device is made difficult by the fact that the `ros2 iron` distribution's package library refers to an older version of `libiio` which doesn't support the serial backend. Thankfully, `libiio`'s transport-layer-agnostic nature means we can still make a connection between the ROS 2 node and the device by launching `iio daemon` on the host to act as a middleman between the serial device and the ROS 2 container, communicating over TCP as a networked `iio` context.

Building on top of the `libiio` wrapper class, we develop a ROS 2 node that communicates with the `iio daemon`, buffering incoming data, and regularly publishes it to the ROS 2 network. Because these two are implicitly asynchronous and may happen at differing rates, they must be separated into two threads, communicating via a synchronized FIFO.

The Python 3 standard library offers very easy to use functionality for this: the `threading` library allows for one-liner instantiation of the `iio` thread, and the `queue` library implements a thread-safe FIFO, removing the need for hand-implemented thread synchronization mechanisms.

The main thread runs the `rcpy` loop and publishes all queued up samples at a rate of 1kHz:

```
class AD4114ExgPublisher(Node):
    # [...snip...]
    def __init__(self, uri, channels):
        # [...snip...]
        self.buffer = Queue()
        self.timer = self.create_timer(1e-3, self.
publish_buffered_samples)
        # [...snip...]

    def publish_buffered_samples(self):
        while not self.buffer.empty():
            (timestamp, channel, value) = self.buffer.get()

            msg = ExgSample()
            msg.header.stamp = Time(seconds=timestamp).to_msg()
            msg.value = value
            self.publisherz[channel].publish(msg)
```

Concurrently, the iio thread connects to the iio daemon, receives samples in blocks, and pushes them to the common buffer. Because libiio transfers are done in chunks, and not individual samples, it would be unrepresentative of the data to assign all samples the timestamp of their arrival or of their request. Similarly, we cannot dead reckon based on the expected sample rate because of the significant $\pm 2.5\%$ accuracy of the AD4114 internal clock. As such, we decided to better trust the host computer's clock and use Python's `time.time()` function, which is documented to use the most precise available clock source. To compute the individual sample timestamps, we linearly interpolate between the timestamp at the end of the previous buffer and the timestamp at the end of the current buffer. Denoting the previous buffer time t_0 (at which point the current buffer started being acquired) and the current buffer time t_1 , the number of samples per buffer N , the timestamp of the k -th sample is estimated as:

$$t[k] = t_0 + \frac{(t_1 - t_0) \cdot k}{N}, \quad k = 0 \dots N - 1 \quad (3.7)$$

A simplified version of the receive thread code (not handling edge cases) is as follows:

```
def receive_thread(self, channels):
    self.iiodev.start(channels)

    t0 = time()
    while True:
        data = self.iiodev.rx()
        t1 = time()

        N = data[0].size
        for i, chan in enumerate(channels):
            for j, value in enumerate(data[i]):
                value = (value - 2**23) * 3e-6
                t = t0 + (t1 - t0) * j / N
                self.buffer.put((t, i, value))

    t0 = t1
```

4 Results and validation

Preamplifier module

We evaluate the designed preamplifier module's performance using two tests based on the ADALM2000 platform, which powers the module, generates test signals, and measures the amplified output.

For the first test, a sinusoidal $A_{test} = 50\text{mV}$ peak-peak test signal is applied to the module's input. The measured output waveform has an expected DC bias of $V_{amp} = 1.88\text{V}$, and an amplitude of $A_{amp} = 1.23\text{V}$ peak-peak, giving an effective gain of $G = A_{amp}/A_{test} = 24.6$. This is just within the 10% tolerance of the designed $G = 22.45$ and can be justified by the tolerances of the passive components we used. For these amplitudes, the harmonic distortion and amplifier noise are not apparent above the -80dBFS noise floor of the ADALM2000.

Secondly, we test the frequency response using the Network analyzer feature of the Scopy software[33]. A $10\text{Hz} - 10\text{kHz}$ sine sweep is generated for the amplifier's input, and the resulting output is measured, determining both the magnitude and phase relation between the two, which are plotted in figure 4.1. Both the measured magnitude and phase characteristics match the ideal frequency response well within the 10% tolerances of the passives.

Above 1kHz , the signal is filtered down so much that measurement noise becomes critical. Increasing amounts of quantization noise result in erroneous measurements: The magnitude plot has an apparently increased magnitude, and the phase plot has considerable amounts of phase noise. This isn't concerning, as it is a limitation of the ADALM2000 used for the test setup, not the preamplifier board itself. More importantly, the desired amplification and corner frequency are validated.

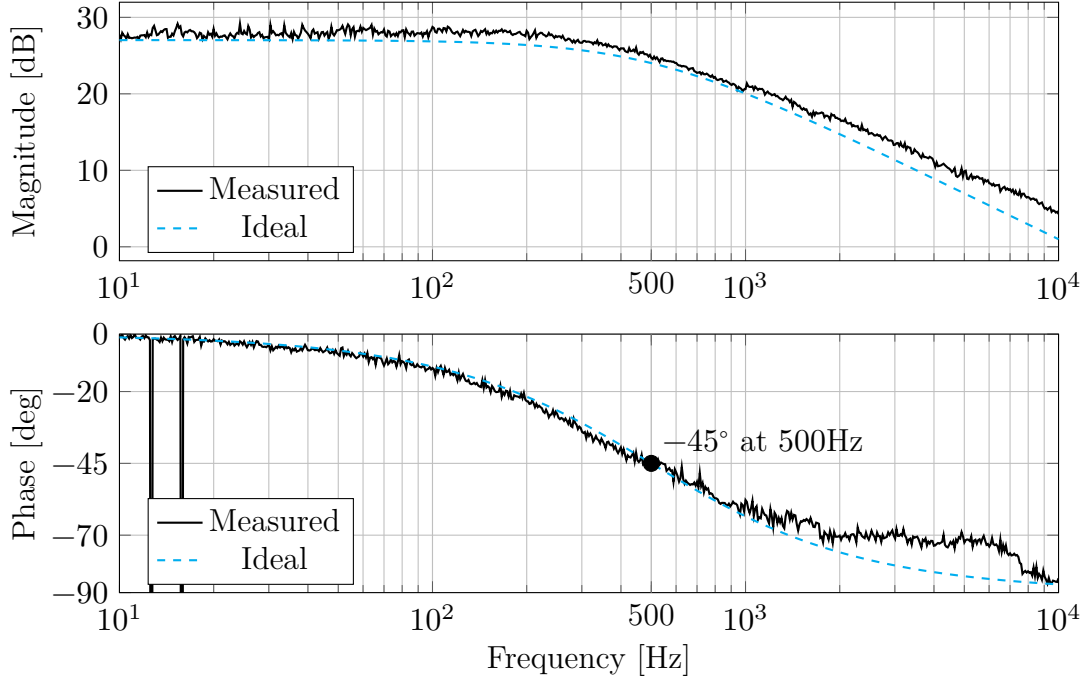


Figure 4.1. Measured preamplifier frequency response, compared to ideal component $+27\text{dB}$ gain, 500Hz first order LPF amplifier

Multi-channel EMG

EMG signals have amplitudes above the circa 1mV noise threshold which is characteristic of max data rate conversions. Thus, measuring 16 single-ended channels can be done without using 16 preamplifier boards, albeit at a loss of signal quality. For evaluation, we do a 6 channel measurement on the inner forearm, targeting multiple muscles used in wrist and finger flexion, and do an exercise with both low effort and high effort movements, to observe both individual action potentials, as well as complex MUAPs. (figure 4.2)

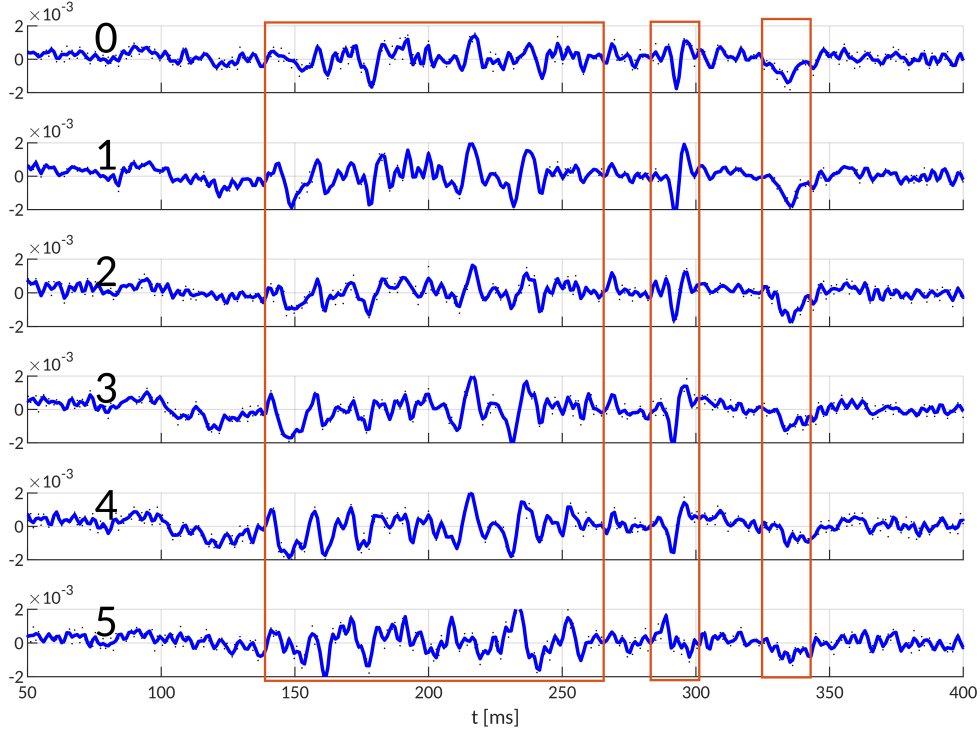


Figure 4.2. 6 Channel EMG measurement without preamplification, displaying an initial period of inactivity, a period of high effort movement (first box), and two individual action potentials during low effort movement (second and third boxes)

In the frequency domain, the highest signal component is -73dB V at 110Hz , and the noise floor is around -85dB V , for an SNR of 12dB without preamplification.

Multi-channel ECG

As proof-of-concept for ECG, we set up a measurement for ECG Leads I (Right Arm - Left Arm), II (Right Arm - Left Leg), III (Left Arm - Left Leg), referenced to the Right Leg through the Driven Right Leg circuit. A 5 second measurement was taken without preamplification and filtered using the MATLAB Signal Processing Toolbox `lowpass` function.

Because the ECG signal has a more restrained frequency composition known a-priori, we take the liberty of using more aggressive digital filtering to significantly improve signal quality. Before filtering, the noise floor lied around -110dB V and the lead II signal's peak was -90dB V at 13.5Hz for an SNR of 20dB . After filtering, the noise floor dropped to -160dB V for an improved SNR of 70dB .

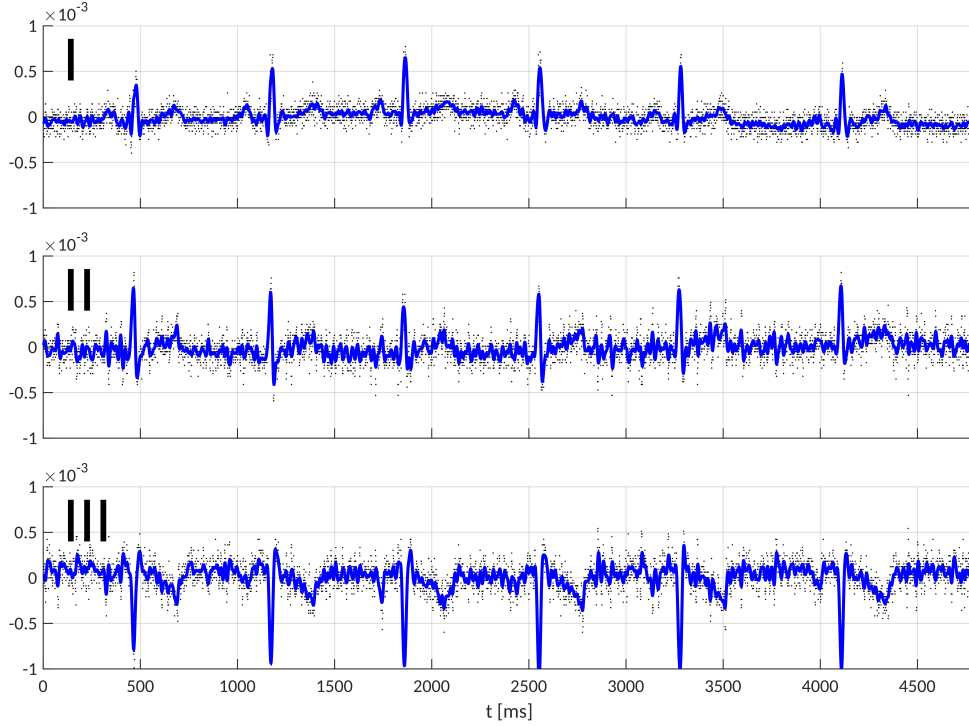


Figure 4.3. ECG leads I, II, III measurement without preamplification

Single-channel ECG, with preamplifier

To demonstrate both the preamplifier and a best-case scenario for ADC SNR, we measure a single channel ECG of the V1 lead. The positive electrode is placed vertically between the 4th and 5th rib, 4cm to the left of the sternum, and the negative electrode on the back of the patient, approximately symmetrical to the positive electrode relative to the heart. The measured signal has a peak of -48dB V at 2Hz and noise floor at -120dB V , resulting in an SNR of 72dB before digital processing. This is much better than the previous measurements and is on par with high-performance commercial systems.

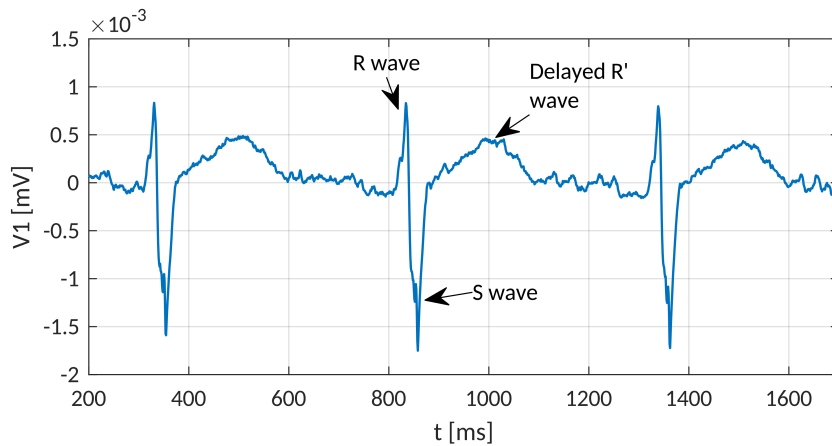


Figure 4.4. Single channel ECG measurement of lead V1 with preamplifier stage

Of morbid curiosity, this ECG (figure 4.4) displays two R waves, the normal first peak, and a delayed and smoother R' wave $\sim 170\text{ms}$ later, confirming the previously clinically diagnosed

RBBB of the author. A healthy individual without RBBB would only show the first R wave as a peak, followed by the S wave as a valley, and no other features until the following heartbeat.

EEG

While outside of the scope of this project, we daringly attempted EEG measurements, inspired by OpenBCI's architecture which supports all three of EMG, ECG and EEG interchangeably. Naive electrode placement and the much weaker nature of EEG signals meant there were no discernable features above the noise floor.

libiio

The data for the previous plots in this section was recorded using iio-oscilloscope[34] which is based on libiio, and later processed and plotted in MATLAB. This proves a complete implementation of all necessary protocol features: selecting channels, assigning inputs to channels, querying DAQ parameters (sample rate, per-channel scale and offset). This validates the libiio component of the project.

ROS 2

We test the ROS 2 publisher node by monitoring its output using plotjuggler[35]. This is a stand-in for any other subscriber node that would further process the data, and allows us to visually inspect the measured waveforms and confirm they match expectations.

One major feature to be evaluated is whether the ROS 2 framework can properly handle the maximum data throughput of this system. The worst case is when measuring 7 to 16 channels with a total data rate of 10309Hz divided between them.

A screenshot of an example EMG measurement in this worst-case configuration is shown in figure 4.5.

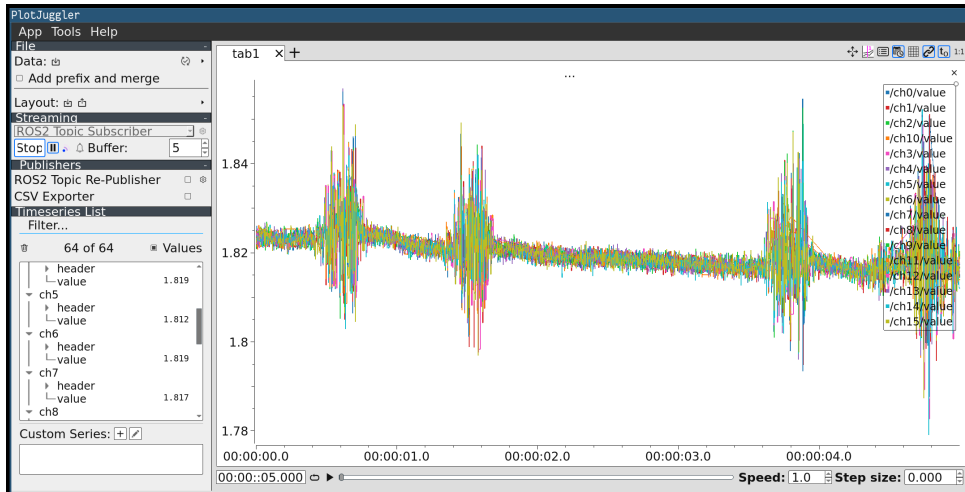


Figure 4.5. ROS 2 Plot Juggler displaying measured and published 16 channel EMG at a per-channel data rate of $10309\text{Hz}/16 = 644\text{Hz}$

5 Conclusion

We present a hardware + software solution intended for easy and affordable maintainability, using off-the-shelf components and open source hardware and software, capable of doing biopotential measurements of up to 16 single-ended or differential channels, with $> 1000\text{Hz}$ sample rates for at most 6 channels and $> 600\text{Hz}$ sample rate for all 16 channels. Both ECG and EMG setups have been validated, with satisfactory results.

The hardware consists of a MAX78000FTHR microcontroller board functioning as a serial libiio device, communicating with an EVAL-AD4114-SDZ ADC board. We designed, built and validated two optional modules: an active Right Leg Drive module, and a differential amplifier module with a gain of $+27\text{dB}$ and active low pass filtering with a cutoff of 500Hz .

The firmware is based on the standard no-OS IIO daemon, with a custom device definition tailored to the use-case of biopotential measurements, allowing for the configuration of desired input channels and making opinionated choices for the ADC configuration in order to maximize signal quality.

On the software side, the system can be interacted with directly via libiio. We further developed python bindings, and a ROS 2 publisher node broadcasting live sample data. We did not focus on automatic interpretation of the signals, but the modular nature of this project allows for easy interconnection with software DSP projects doing that.

Evaluating the established objectives, we check all MUSTs, and 3/5 SHOULDs. This implementation falls short on the available data rate when activating more than 10 channels, with a max ODR of 10309Hz divided among all channels, as well as on the precision requirement, being limited to $3\mu\text{V}$ resolution without the preamplifier board, but having sub-microvolt precision with it.

Future work includes:

- Upgrading the AD4114 to another ADC in the AD717X family, such as the better-performing AD7175-8 with up to 50kHz ODR for 16 channels
- Designing and testing a dedicated PCB integrating the whole system, instead of using interconnected modules. This may further improve signal quality, and cost.
- Developing the software stack to have more capabilities than just displaying waveforms

Acknowledgements

I would like to sincerely thank my thesis coordinator Tassos Natsakis for his guidance and unending patience, my colleague Mark Thoren for assisting with mixed-signal system design resources, and my manager Monica Ignat for kindly allowing me the flexibility I needed to finish this project.

6 Bibliography

- [1] M. Barbero, R. Merletti, and A. Rainoldi, *Atlas of Muscle Innervation Zones*. Springer Milan, 2012, ISBN: 9788847024632. DOI: 10.1007/978-88-470-2463-2. [Online]. Available: <http://dx.doi.org/10.1007/978-88-470-2463-2>.
- [2] *Robot operating system (ROS) 2 documentation*, <https://docs.ros.org/en/iron/index.html>, Accessed: 2024-08.
- [3] W. J. Esposito, “The ASPEN platform : Tools for mixed signal electronics, digital signal processing, and biomedical electronics,” PhD dissertation, Department of Electrical Engineering, Stanford University, CA, USA, 2018. [Online]. Available: <https://purl.stanford.edu/nq977kw3386>.
- [4] *OpenBCI - Open Source Biosensing Tools (EEG, EMG, EKG, and more)*, <https://openbci.com/>, Accessed: 2024-06.
- [5] *BioAmp EXG pill*, <https://github.com/upsidedownlabs/BioAmp-EXG-Pill>, Accessed: 2024-06.
- [6] *Myoware muscle sensors*, <https://www.sparkfun.com/myoware>, Accessed: 2024-06.
- [7] P. M. Hopkins, “Skeletal muscle physiology,” *Continuing Education in Anaesthesia Critical Care & Pain*, vol. 6, no. 1, pp. 1–6, Feb. 2006. DOI: 10.1093/bjaceaccp/mki062. [Online]. Available: <https://doi.org/10.1093/bjaceaccp/mki062>.
- [8] R. E. Klabunde, *Cardiovascular Physiology Concepts*, 2nd ed. Philadelphia, PA: Lippincott Williams and Wilkins, Sep. 2011.
- [9] E. Henneman, “Relation between size of neurons and their susceptibility to discharge,” *Science*, vol. 126, no. 3287, pp. 1345–1347, 1957, ISSN: 00368075, 10959203. Accessed: Aug. 21, 2024. [Online]. Available: <http://www.jstor.org/stable/1752769>.
- [10] C. J. De Luca, “Control properties of motor units,” *Journal of Experimental Biology*, vol. 115, no. 1, pp. 125–136, Mar. 1985, ISSN: 1477-9145. DOI: 10.1242/jeb.115.1.125. [Online]. Available: <http://dx.doi.org/10.1242/jeb.115.1.125>.
- [11] S. Meek, “Abc of clinical electrocardiography: Introduction. i—leads, rate, rhythm, and cardiac axis,” *BMJ*, vol. 324, no. 7334, pp. 415–418, Feb. 2002, ISSN: 1468-5833. DOI: 10.1136/bmj.324.7334.415. [Online]. Available: <http://dx.doi.org/10.1136/bmj.324.7334.415>.
- [12] S. Lee and J. Kruse, “Biopotential electrode sensors in ECG/EEG/EMG systems,” 2008. [Online]. Available: https://www.analog.com/media/en/technical-documentation/technical-articles/ECG-EEG-EMG_FINAL.pdf.
- [13] M. S. Lee, A. Paul, Y. Xu, W. D. Hairston, and G. Cauwenberghs, “Characterization of ag/agcl dry electrodes for wearable electrophysiological sensing,” *Frontiers in Electronics*, vol. 2, Jan. 2022, ISSN: 2673-5857. DOI: 10.3389/felec.2021.700363. [Online]. Available: <http://dx.doi.org/10.3389/felec.2021.700363>.
- [14] M. Seiti, A. Giuri, C. E. Corcione, and E. Ferraris, “Advancements in tailoring pedot: Pss properties for bioelectronic applications: A comprehensive review,” *Bio-materials Advances*, vol. 154, p. 213655, Nov. 2023, ISSN: 2772-9508. DOI: 10.1016/

- j.bioadv.2023.213655. [Online]. Available: <http://dx.doi.org/10.1016/j.bioadv.2023.213655>.
- [15] S. Hou et al., “Highly stretchable self-adhesive pedot:pss dry electrodes for biopotential monitoring,” *ACS Applied Polymer Materials*, Jun. 2024, ISSN: 2637-6105. DOI: 10.1021/acsapm.4c01233. [Online]. Available: <http://dx.doi.org/10.1021/acsapm.4c01233>.
- [16] *Cometa pico emg*, <https://www.cometasystems.com/picox/>, Accessed: 2024-07.
- [17] *Trigno avanti sensor, delsys europe*, <https://delsyseurope.com/trigno-avanti/>, Accessed: 2024-08.
- [18] S. Fuentes del Toro, Y. Wei, E. Olmeda, L. Ren, G. Wei, and V. Diaz, “Validation of a low-cost electromyography (emg) system via a commercial and accurate emg device: Pilot study,” *Sensors*, vol. 19, p. 5214, Nov. 2019. DOI: 10.3390/s19235214.
- [19] V. J. Urick, K. J. Williams, and J. D. McKinney, *Fundamentals of microwave photonics* (Wiley Series in Microwave and Optical Engineering), en. Nashville, TN: John Wiley & Sons, Feb. 2015.
- [20] F. Suarez, A. Nozariasbmarz, D. Vashae, and M. C. Öztürk, “Designing thermoelectric generators for self-powered wearable electronics,” *Energy & Environmental Science*, vol. 9, no. 6, pp. 2099–2113, 2016, ISSN: 1754-5706. DOI: 10.1039/c6ee00456c. [Online]. Available: <http://dx.doi.org/10.1039/C6EE00456C>.
- [21] J. F. Kurniawan et al., “Electrochemical performance study of ag/agcl and au flexible electrodes for unobtrusive monitoring of human biopotentials,” *Nano Select*, vol. 3, no. 8, pp. 1277–1287, May 2022, ISSN: 2688-4011. DOI: 10.1002/nano.202100345. [Online]. Available: <http://dx.doi.org/10.1002/nano.202100345>.
- [22] *Understanding and eliminating 1/f noise*, <https://www.analog.com/en/resources/analog-dialogue/articles/understanding-and-eliminating-1-f-noise.html>, Accessed: 2024-08.
- [23] S. Chatterjee, R. S. Thakur, R. N. Yadav, L. Gupta, and D. K. Raghuvanshi, “Review of noise removal techniques in ecg signals,” *IET Signal Processing*, vol. 14, no. 9, pp. 569–590, Dec. 2020, ISSN: 1751-9683. DOI: 10.1049/iet-spr.2020.0104. [Online]. Available: <http://dx.doi.org/10.1049/iet-spr.2020.0104>.
- [24] A. B. Brad Brannon, “AN-501: Aperture uncertainty and adc system performance,” 2006. [Online]. Available: <https://www.analog.com/media/en/technical-documentation/application-notes/an-501.pdf>.
- [25] *LibIIO internals*, https://wiki.analog.com/resources/tools-software/linux-software/libiio_internals, Accessed: 2024-08.
- [26] *8-channel, low noise, low power, 24-bit, sigma-delta ADC with PGA and reference*, AD7124-8, Rev. F, Analog Devices, 2023. [Online]. Available: <https://www.analog.com/media/en/technical-documentation/data-sheets/ad7124-8.pdf>.
- [27] *Single supply, multichannel, 31.25 kSPS, 24-bit, sigma-delta ADC with ± 10 V inputs*, AD4114, Rev. A, Analog Devices, 2020. [Online]. Available: <https://www.analog.com/media/en/technical-documentation/data-sheets/ad4114.pdf>.

- [28] *Wide supply range, rail-to-rail output instrumentation amplifier*, AD8226, Rev. D, Analog Devices, 2019. [Online]. Available: <https://www.analog.com/media/en/technical-documentation/data-sheets/AD8226.pdf>.
- [29] *No-OS: Software drivers in C for systems without an operating system*, <https://analogdevicesinc.github.io/no-OS/>, Accessed: 2024-08.
- [30] *Ad717x no-os software drivers*, <https://wiki.analog.com/resources/tools-software/uc-drivers/ad717x>, Accessed: 2024-06.
- [31] D. B. de Oliveira, D. Casini, R. S. de Oliveira, and T. Cucinotta, “Demystifying the real-time linux scheduling latency,” en, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020. DOI: 10.4230/LIPICS.ECRTS.2020.9. [Online]. Available: <https://drops.dagstuhl.de/entities/document/10.4230/LIPICS.ECRTS.2020.9>.
- [32] S. Smith, “How to initialize and interact with programmable i/o,” in *RP2040 Assembly Language Programming*. Apress, Oct. 2021, pp. 177–200, ISBN: 9781484277539. DOI: 10.1007/978-1-4842-7753-9_10. [Online]. Available: http://dx.doi.org/10.1007/978-1-4842-7753-9_10.
- [33] *Scopy*, <https://wiki.analog.com/university/tools/m2k/scopy>, Accessed: 2024-06.
- [34] *Iio-oscilloscope: A GTK+ based oscilloscope application for interfacing with various IIO devices*, <https://github.com/analogdevicesinc/iio-oscilloscope>, Accessed: 2024-08.
- [35] *Plotjuggler: Fast, intuitive and extensible time series visualization tool*. <https://plotjuggler.io/>, Accessed: 2024-08.